
DTU

62595 Full stack udvikling, drift og distribuerede systemer

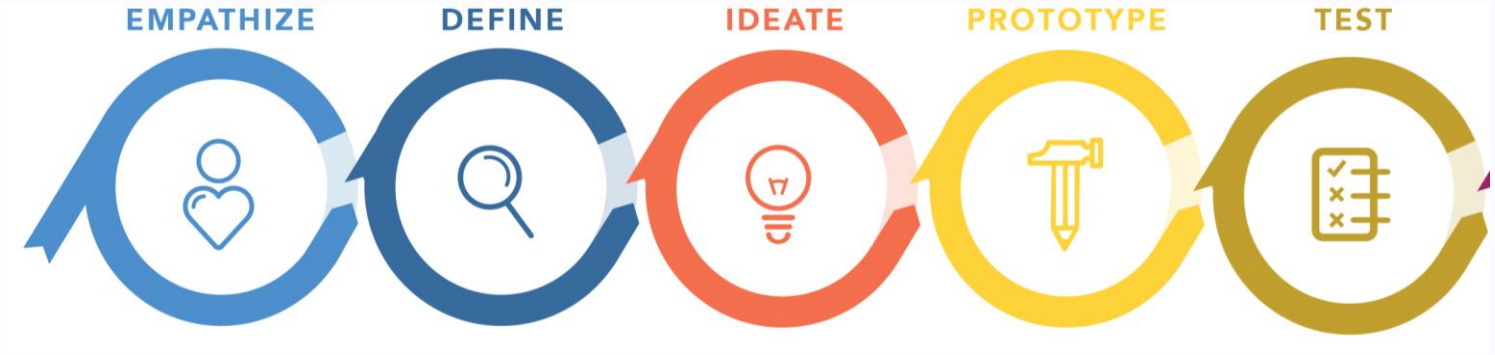
Fra Design til udvikling

- ❑ Opsummering
- ❑ InfoVis
- ❑ Designsystem
- ❑ Agile udvikling

Opsummering

Understand

Explore

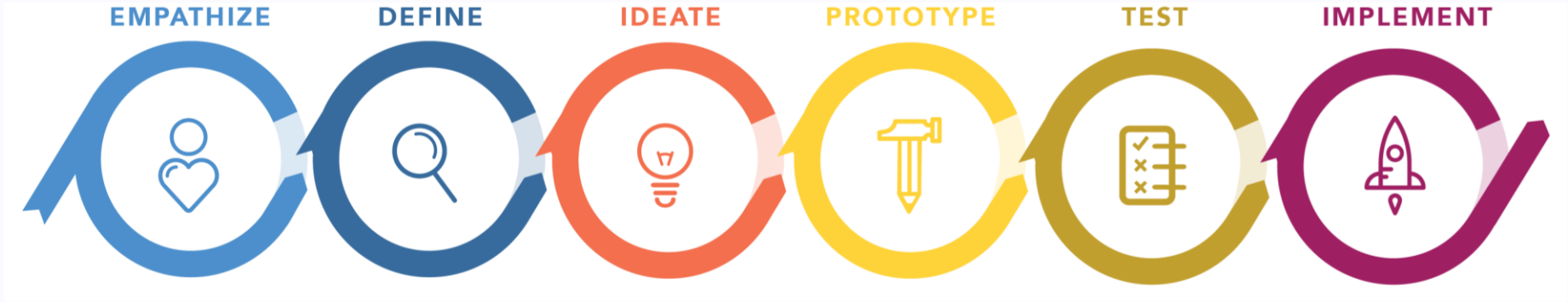


Understand

Explore

Brugerstudie

Materialize



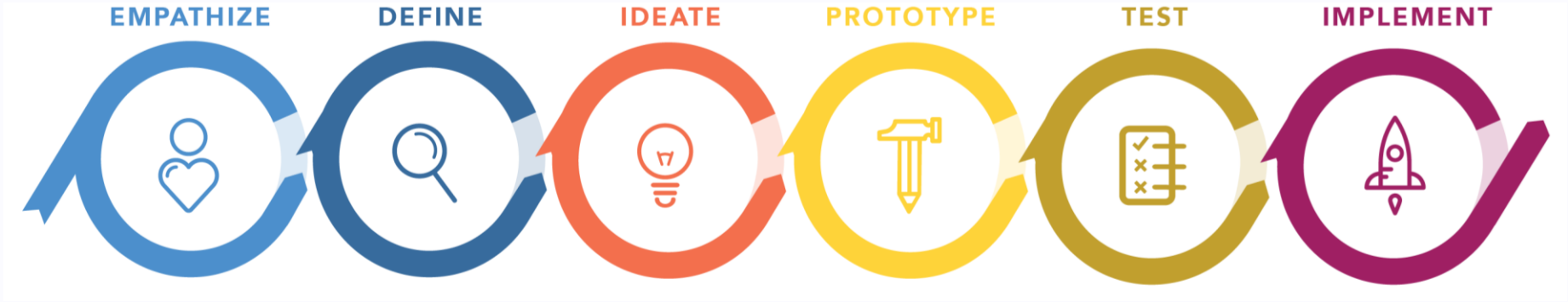
Vi er her

Understand

Explore

Brugerstudie

Materialize





https://www.codecademy.com/resources/docs/uiux/think-aloud-protocol

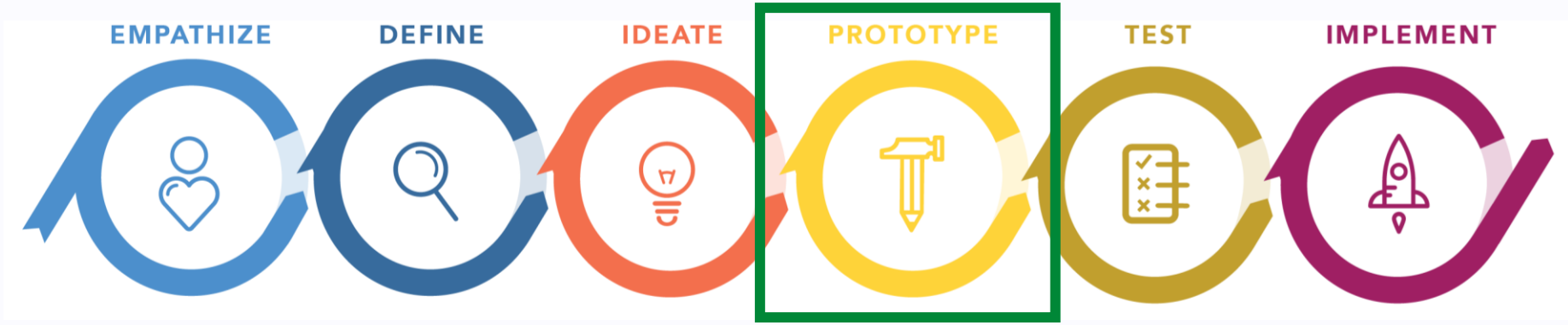
Vi er her

Understand

Explore

Brugerstudie

Materialize



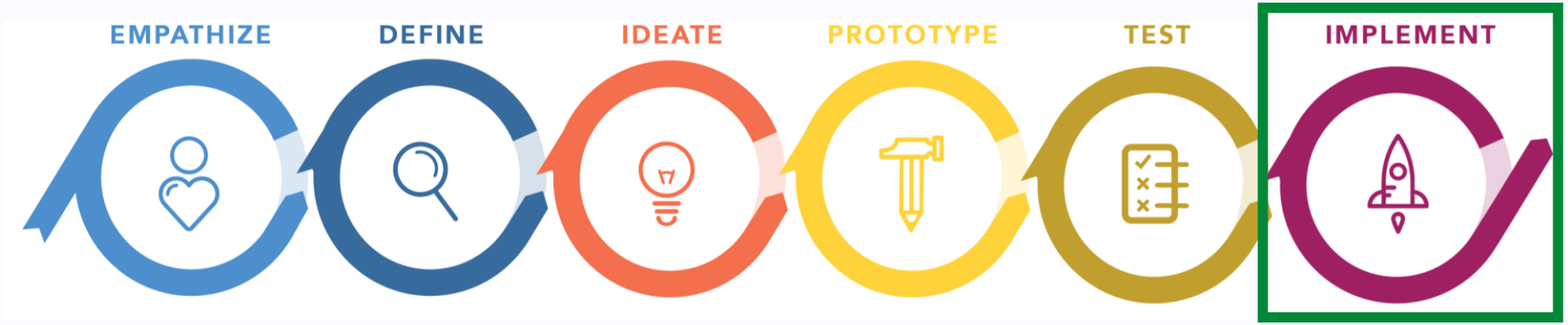
Vi er her

Understand

Explore

Brugerstudie

Materialize



InfoVis 101

InfoVis 101

- Kort introduktion til InfoVis
- Fokus på visuel forståelse og mobile enheder
- Senere præsentation om plots i kotlin følger op på det der introduceres i dag

arXiv:2604.08378v1 [cs.GR] 9 Apr 2026

INVESTIGATING PERFORMANCE AND PRACTICES WITH UNIVARIATE DISTRIBUTION CHARTS

Laura Lotteraner
University of Vienna
Vienna, Austria
laura.lotteraner@univie.ac.at

Anna Kurtenkova
University of Vienna
Vienna, Austria

Torsten Möller
University of Vienna
Vienna, Austria

Daniel Pahr
University of Vienna
Vienna, Austria

April 10, 2026

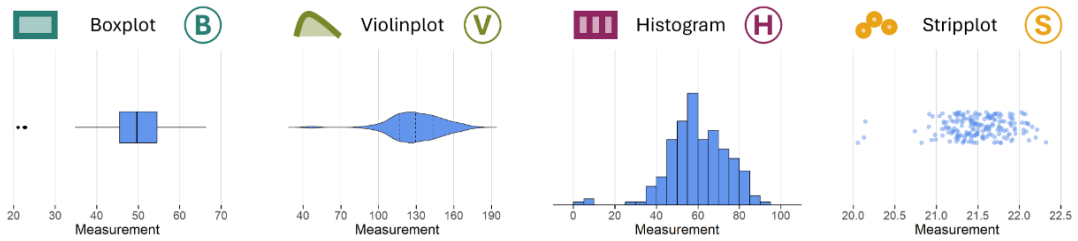


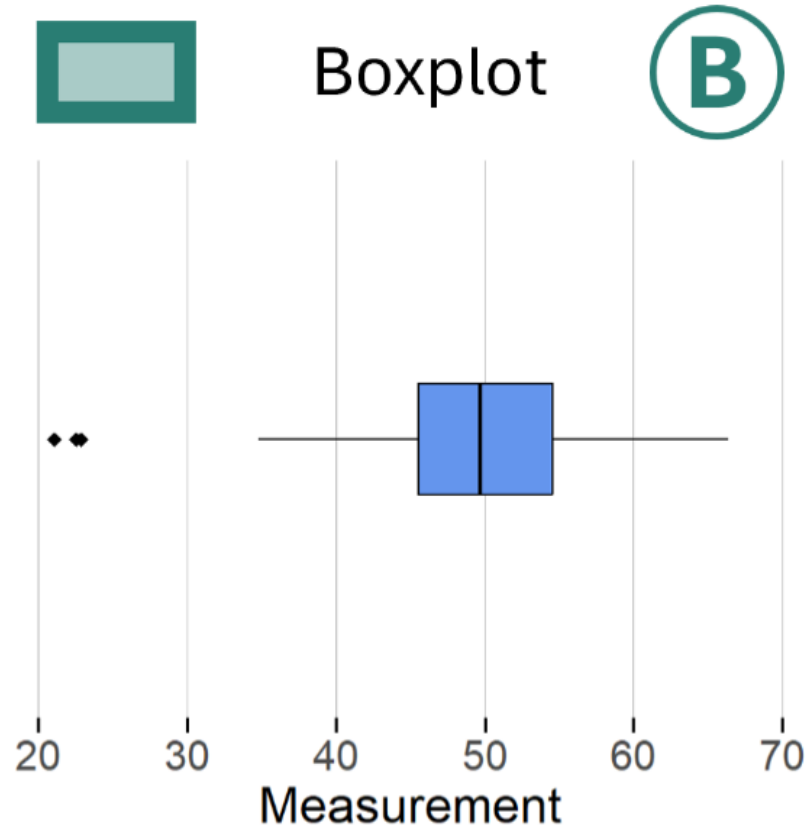
Figure 1: *The four charts used in this study (dataset 1):* **Boxplot**, **Violinplot**, **Histogram**, *and* **Stripplot**, *each representing one of the four classes of charts:* *Summary Statistics*, *Smooth Densities*, *Binned Densities*, *Individual Data Points.*

ABSTRACT

A range of charts with different strengths and weaknesses exists to support the visual analysis of univariate distributions, with a limited understanding of which charts best support which tasks and users, and how practitioners use charts. We categorize the available charts for univariate distributions into four groups and present the results of a mixed-methods comparison (n=215) of participants' perception and preferences across boxplots, violinplots, jittered stripplots, and histograms as representatives of their respective categories. The click-to-select approach in our study, combined with data on participants' subjective experiences and preferences, allows to both measure accuracy on benchmark tasks and discuss participants' choices qualitatively.

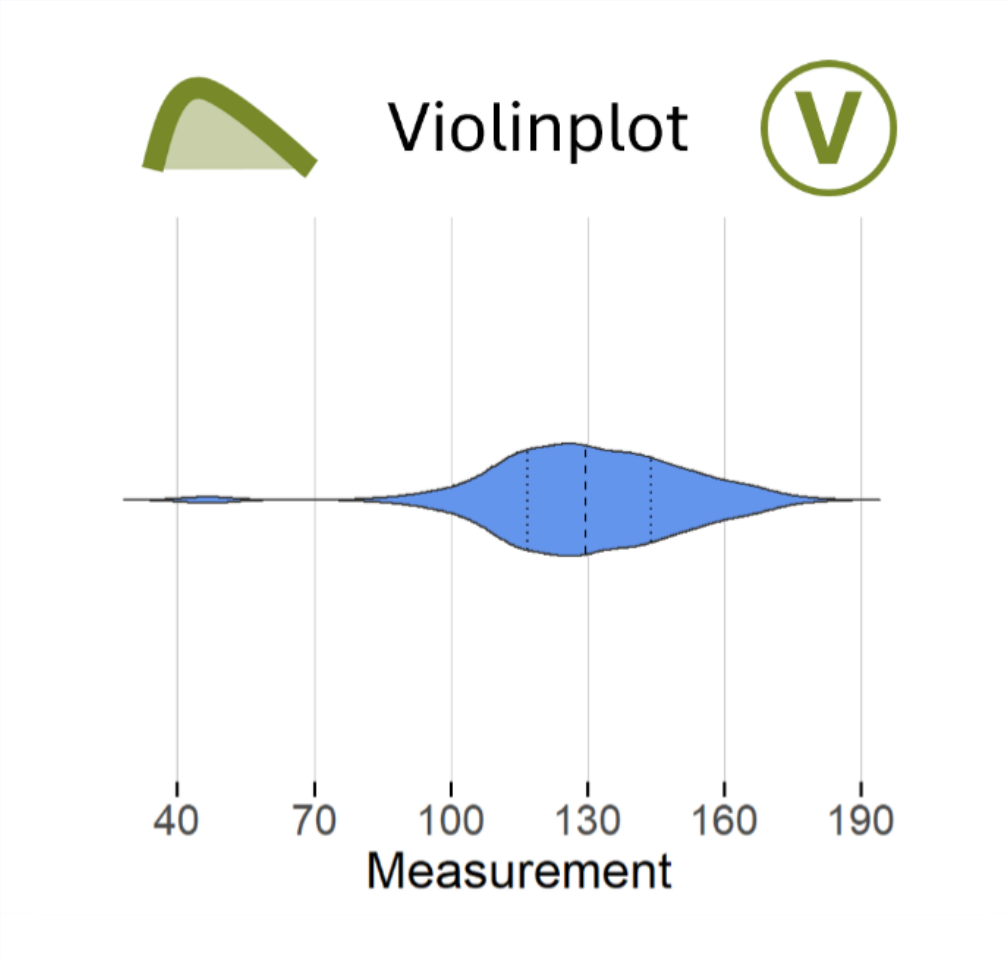
Our analysis reveals differences between charts in task accuracy, common misunderstandings, and preferences across various low-level tasks, and indicates that chart preference and familiarity do not necessarily align with participants' task performance. Interviews with five visualization practitioners further reveal that charts widely preferred by general audiences (such as histograms) or commonly used in scientific domains (such as boxplots) are not inherently the most effective for all tasks.

Forstår brugeren egentlig din graf?



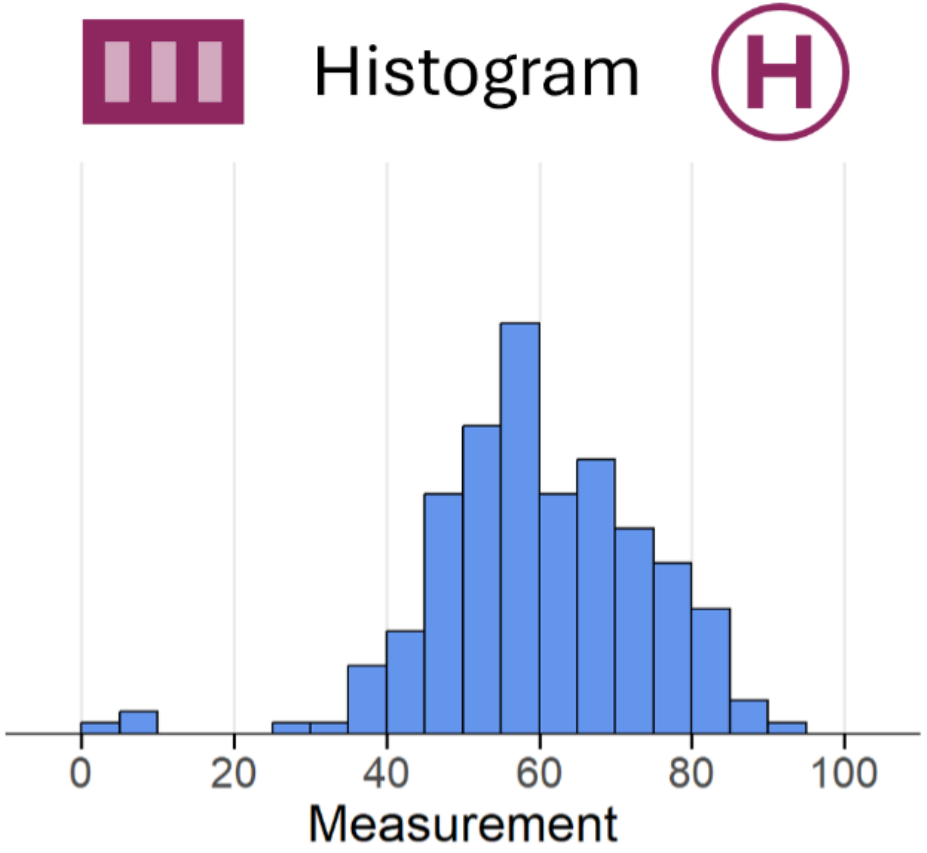
- Standard i mange videnskabelige domæner og skabt til hurtig sammenligning af opsummerende statistik (som medianen).
- **Svaghed:** Har et meget højt abstraktionsniveau, og formen på selve fordelingen skjules.
- **Brugerforståelse:** Forskningen viser, at abstrakte elementer som selve "boksen" er yderst tilbøjelige til at blive misforstået af almindelige brugere.

Forstår brugeren egentlig din graf?



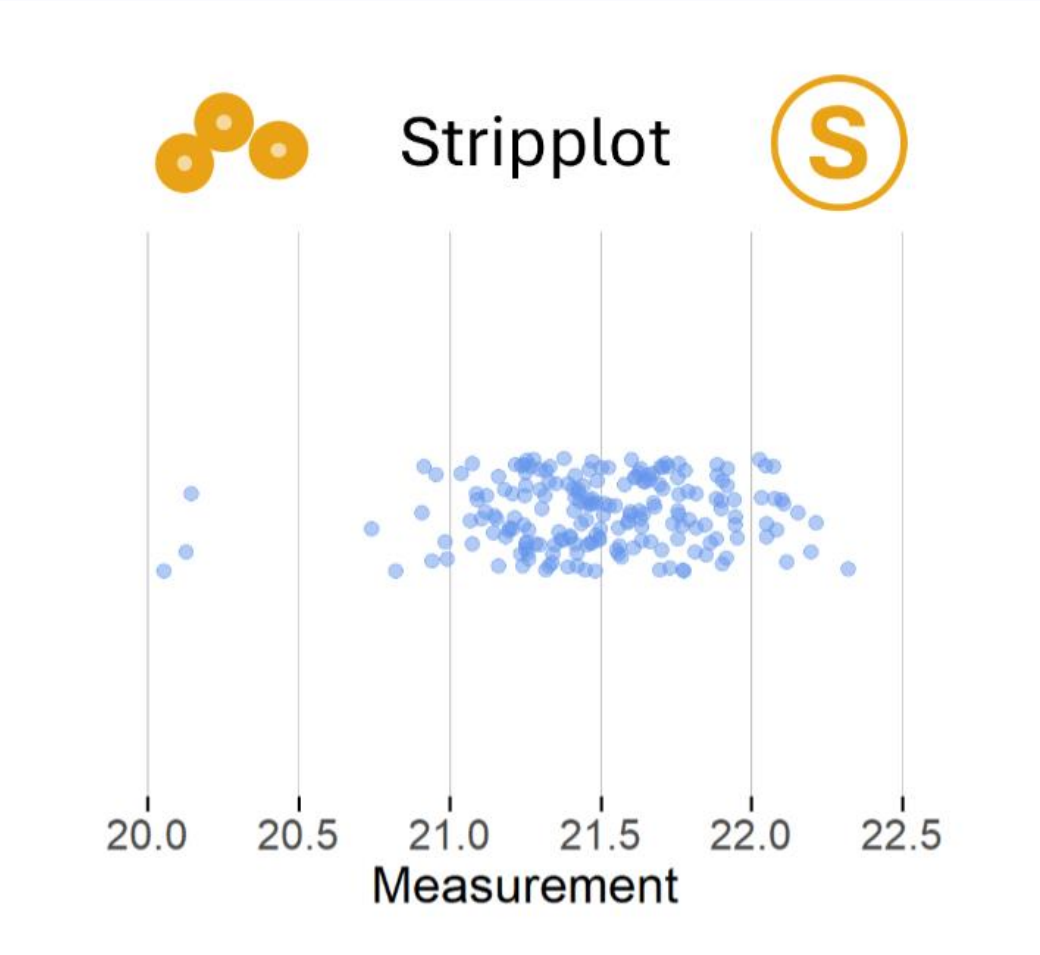
- Viser en udglattet kurve (smooth density) af dataens fordeling, spejlet symmetrisk.
- **Svaghed:** Den udglattede kurve kan sløre den præcise rækkevidde (range) af dataen.
- Selvom brugerne var mindst fortrolige med grafen, fandt de dens bløde kurver meget intuitive til at spotte mønstre, "clusters" (klynger) og fordelingens form.

Forstår brugeren egentlig din graf?



- Viser data samlet i "kasser" (binned densities) og er den mest kanoniske måde at vise fordelinger på.
- **Svaghed:** Eksperter undgår dem ofte til avanceret analyse, da de fylder meget og gør det svært at sammenligne mange forskellige grupper på én gang.
- Deltagerne i studiet følte sig mest selvsikre med histogrammer og foretrak dem generelt over alle andre typer på grund af høj genkendelighed. **(Publikumsfavoritten)**

Forstår brugeren egentlig din graf?



- Viser de individuelle, rå datapunkter, spredt ud for at undgå at de ligger oven i hinanden.
- **Svaghed:** Ved store datasæt forhindrer grafen evnen til at udpege relevante områder, den opfattes som meget uoverskuelig ("cluttered"), og brugerne fandt den uintuitiv.
- Roset for at være gennemsigtig og meget brugbar til at identificere datasættets spændvidde, især for små datasæt.

Takeaways

- **Præference ≠Præstation:** Brugere foretrækker ofte det, de kender (histogrammer), men løser nogle opgaver mere præcist med andre grafer.
- **Abstraktion vs. Forvirring:** Eksplicitte markeringer hjælper, men for meget abstraktion (Boxplot) forvirrer. Rå data skaber tillid, men for meget data (Stripplot) overvælder.
- **Konklusion for jeres app:** Vælg grafen ud fra opgaven og målgruppen, og lad være med at vælge et boxplot, bare fordi det ser "videnskabeligt" ud.

Komparativ Visualisering

Find fem fejl (12)



Find fem fejl (12)



Taksonomi for komparativ visualisering (Gleicher et al.)

- Juxtaposition (Side-om-side)
- Superposition (Overlejring i samme graf)
- Eksplicit visning af relationer (f.eks. forskellen)

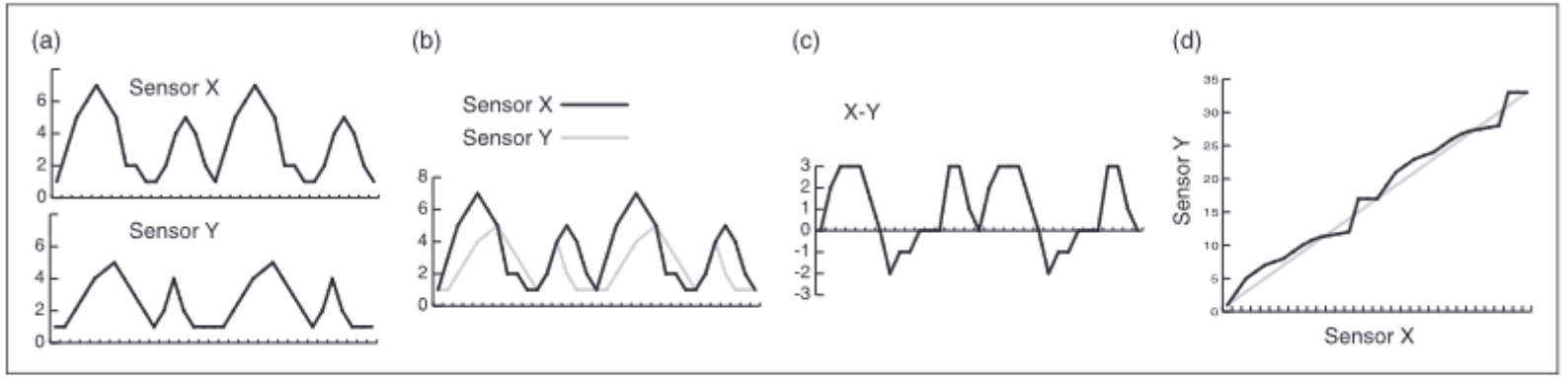


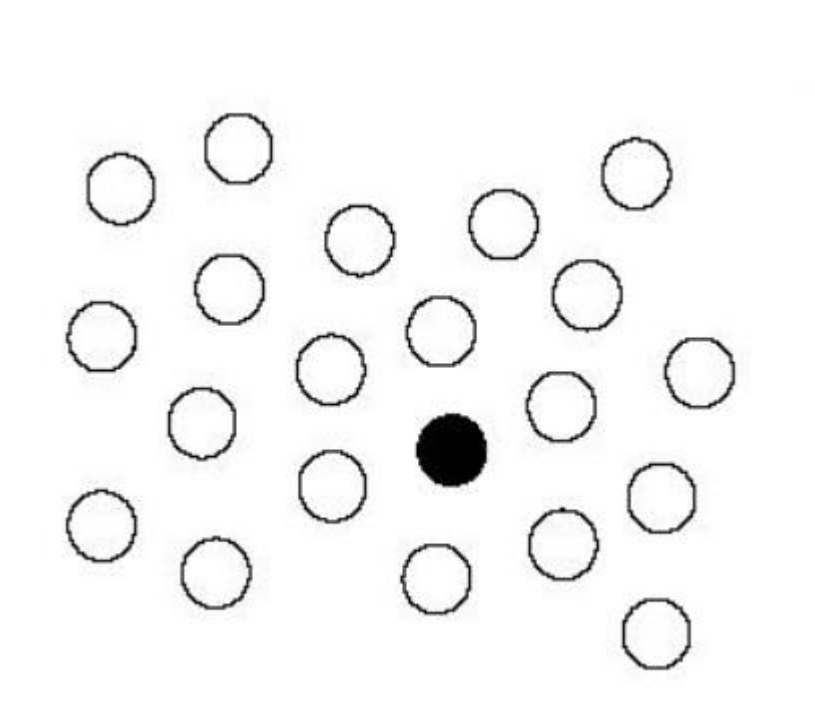
Figure 1. A simple example of comparative visualizations: comparing two time series. For example, the data might represent measurements from two sensors (X and Y) taken over time. The three basic approaches for comparative visualization are (a) juxtaposition, (b) superposition, (c,d) explicit encoding of relationships. Two new representations from fusion are shown: (c) signal subtraction and (d) a time alignment curve, such as produced by dynamic time warping. Used with permission.

Hvorfor gøre mere end bare side-om-side?



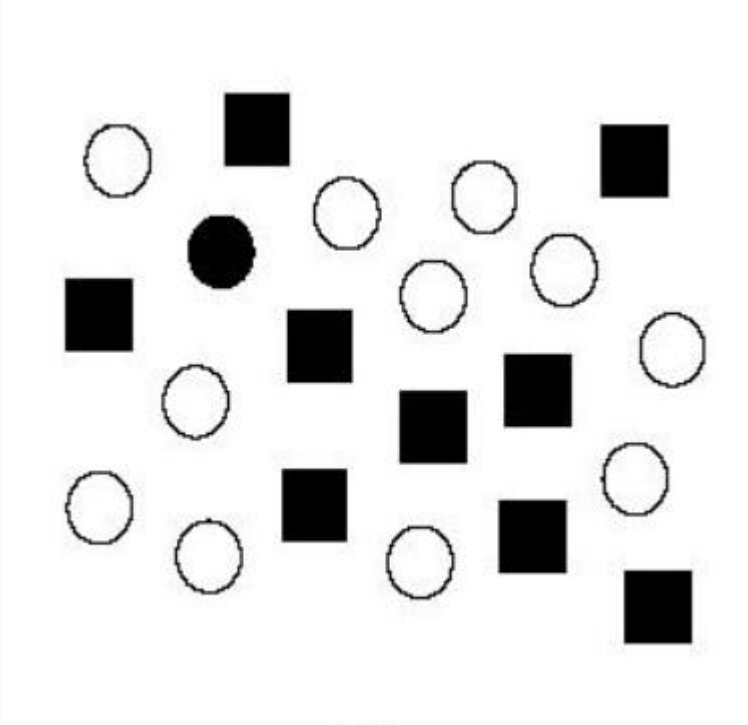
Hvorfor gøre mere end bare side-om-side?

- Præattentiv processering (Ubevidst opfattelse)
  - Præattentive træk inkluderer farvetone, krumning, størrelse, intensitet, orientering, længde osv.



Hvorfor gøre mere end bare side-om-side?

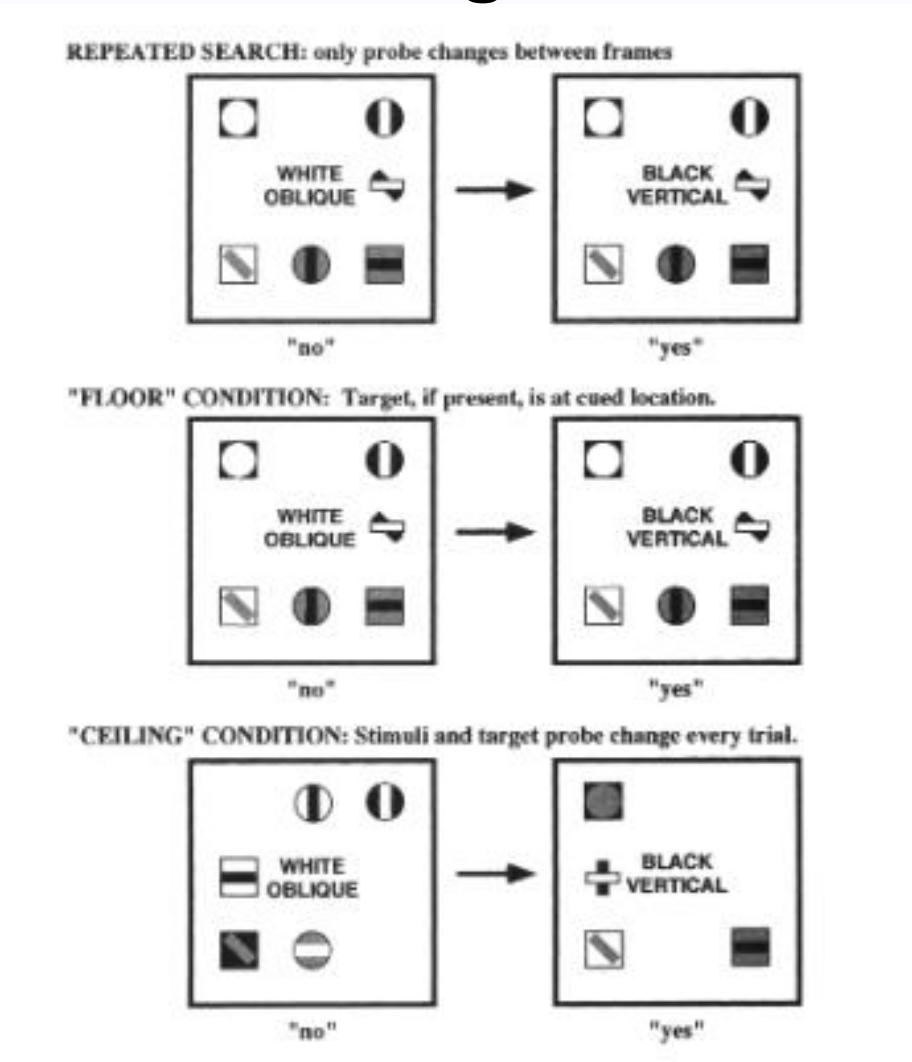
- Præattentiv processering (Ubevidst opfattelse)
  - Præattentive træk inkluderer farvetone, krumning, størrelse, intensitet, orientering, længde osv.
  - Kombinationer af disse kan *ikke* opfattes præattentivt
  - Kan forårsage visuel forvirring (interferens)

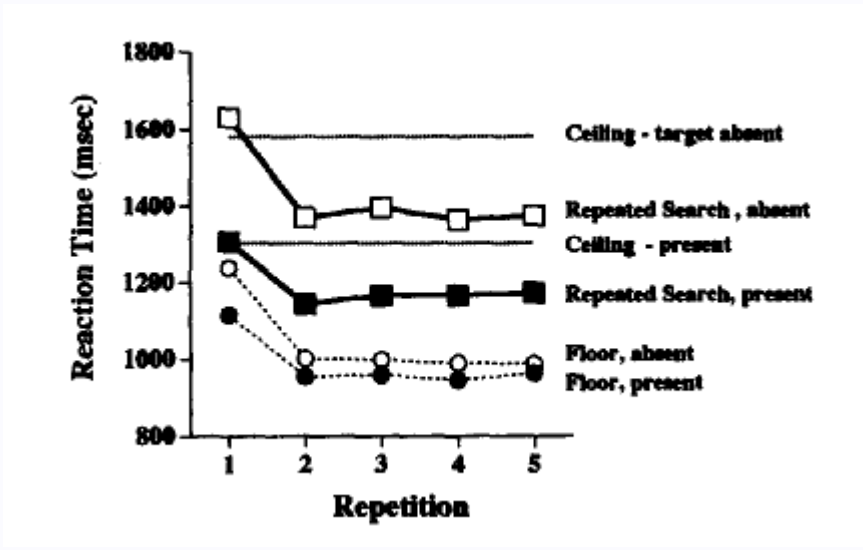


Hvorfor gøre mere end bare side-om-side?

- Postattentivt syn (Bevidst opfattelse)
  - Vi genkender kun ét objekt ad gangen (kobling mellem langtidshukommelse og visuel stimuli)
  - Vi flytter bevidst opmærksomheden mellem objekter
- Kapaciteten for vores visuelle arbejdshukommelse:
  - Maks. 4 objekter med op til 4 træk

Hvorfor gøre mere end bare side-om-side?





Hvorfor gøre mere end bare side-om-side?



Hvorfor gøre mere end bare side-om-side?

- Komplekse og store datasæt
- Eksplorativ dataanalyse
- Forbedrer evnen til at sammenligne datasæt og objekter
- Mere på mindre plads
- Bedre data-to-ink ratio

Beyond One-Size-Fits-All: User Strategies for Simplification Technique and Level Selection in Responsive Line Charts

Rifat Ara Proma and Paul Rosen

Scientific Computing and Imaging Institute, University of Utah, Salt Lake City, Utah, USA

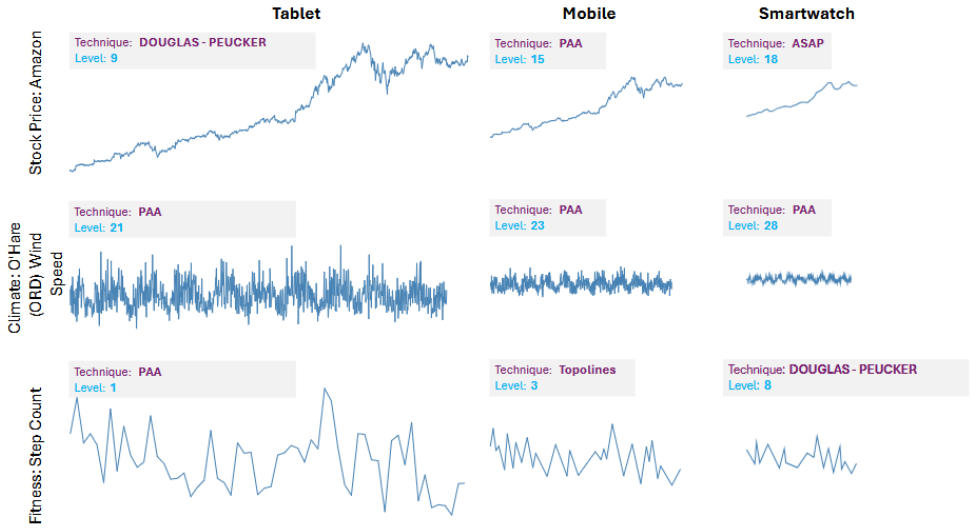


Figure 1: The figure shows the final simplified charts generated by different participants under the C2 condition. Participants selected different simplification techniques and levels depending on the dataset and device size.

Abstract

Simplifying line charts for responsive displays typically applies a single algorithm uniformly across devices, despite the availability of multiple techniques that preserve different signal characteristics (e.g., peaks, trends, periodicity). We investigate whether users benefit from algorithmic choice when adapting charts across screen sizes. In a within-subjects study (N=30), participants simplified nine datasets under three conditions: single pre-assigned technique (C1), multiple techniques (C2), and multiple techniques with manual point selection (C3), each with control over simplification level. We found that users adapted technique selections across datasets rather than devices, leveraging dataset-level strategies rather than per-device optimization. Additionally, interaction complexity did not always increase engagement uniformly, suggesting that responsive simplification tools should balance algorithmic flexibility with progressive disclosure and strong defaults. Supplemental materials are available at https://osf.io/yjp76/?view_only=b77b5e97f0cc4f689fbf48ad0d965af3.

CCS Concepts

• ***Human-centered computing*** → ***Information visualization; Visualization design and evaluation methods;***

Man kan ikke bare “krympe” graferne

- Når data flyttes fra en stor tablet til en lille mobilskærm, opstår der lynhurtigt visuel støj og uoverskueligt rod.
- For at løse dette skal selve den underliggende data forsimples (simplification), f.eks. ved at samle datapunkter eller udglatte kurven.
- Forskning viser, at "One-size-fits-all" ikke fungerer: Forskellige typer data kræver forskellige forsimplingsteknikker.

"One-Size-Fits-All" virker ikke

- Datasættet bestemmer teknikken. Studiet viste, at deltagerne skiftede forsimplings-teknik alt efter, hvilken *type* data de kiggede på.
- Aktier vs. Fitness. For ustabile data som aktiekurser valgte brugerne teknikker (som ASAP og Topolines), der bevarede ekstreme udsving (peaks). For fitness-skridttællere foretrak de teknikker (som Douglas-Peucker), der skabte en glattere, generel trend.
- Brugerne vil have valgmuligheder. Over 56 % af deltagerne foretrak at have adgang til flere forskellige forsimplings-teknikker frem for kun at have én.

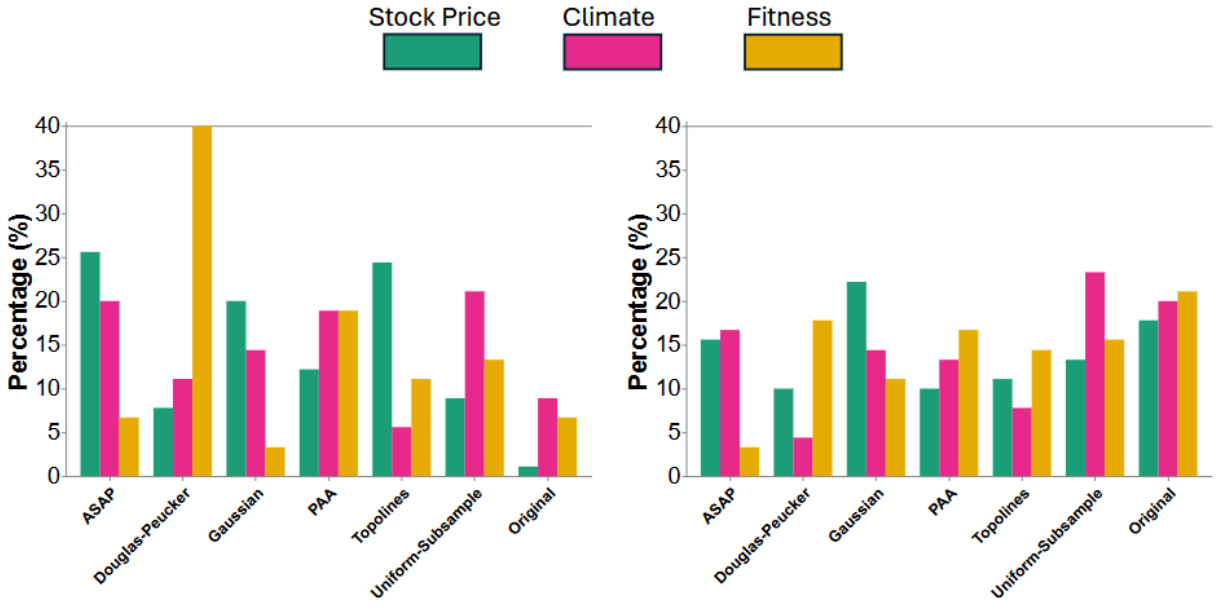


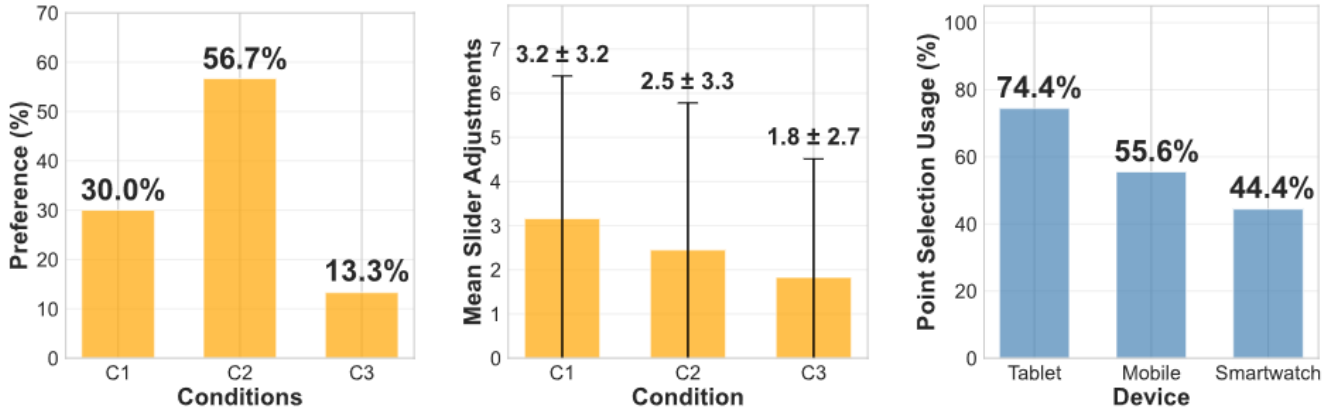
Figure 2: Final simplification technique distribution across dataset categories in C2 (left) and C3 (right). Bars represent the percentage of finalized configurations using each technique, grouped by dataset type (Stock, Climate, Fitness).

Takeaways

- Data-reduktion er et UX-valg. Det er et bevidst designvalg at beslutte, om jeres graf på mobilen skal bevare ekstreme peaks eller vise en glat gennemsnitskurve.
- Tilpas efter domæne. Jeres valg af forsimpling skal matche formålet (skal brugeren se en langsigtet trend eller et pludseligt udsving?).
- Gode standardindstillinger er vigtige. Selvom valgfrihed er godt, viste studiet også, at for meget kompleks manuel kontrol får brugerne til at give op ("disengage"). Sørg for at have stærke "defaults".

Takeaways

- Graf (a) viser, at hele 56,7 % af brugerne foretrak at have adgang til *flere* forskellige forsimplings-teknikker (C2) frem for kun at have én (C1).
- Studiet viser også, at den allermest avancerede løsning(C3), hvor brugerne fik lov at udvælge datapunkter manuelt, var den *mindst* populære med kun 13,3 %. For meget kompleksitet dræber brugeroplevelsen!
- Tilbyd brugerne få, men meningsfulde valg (f.eks. "Vis detaljer" vs. "Vis gennemsnit"), men sørg altid for at have stærke standardindstillinger ("defaults") i jeres design.



(a) Participant preference (b) Slider adjustments (c) Point selection usage

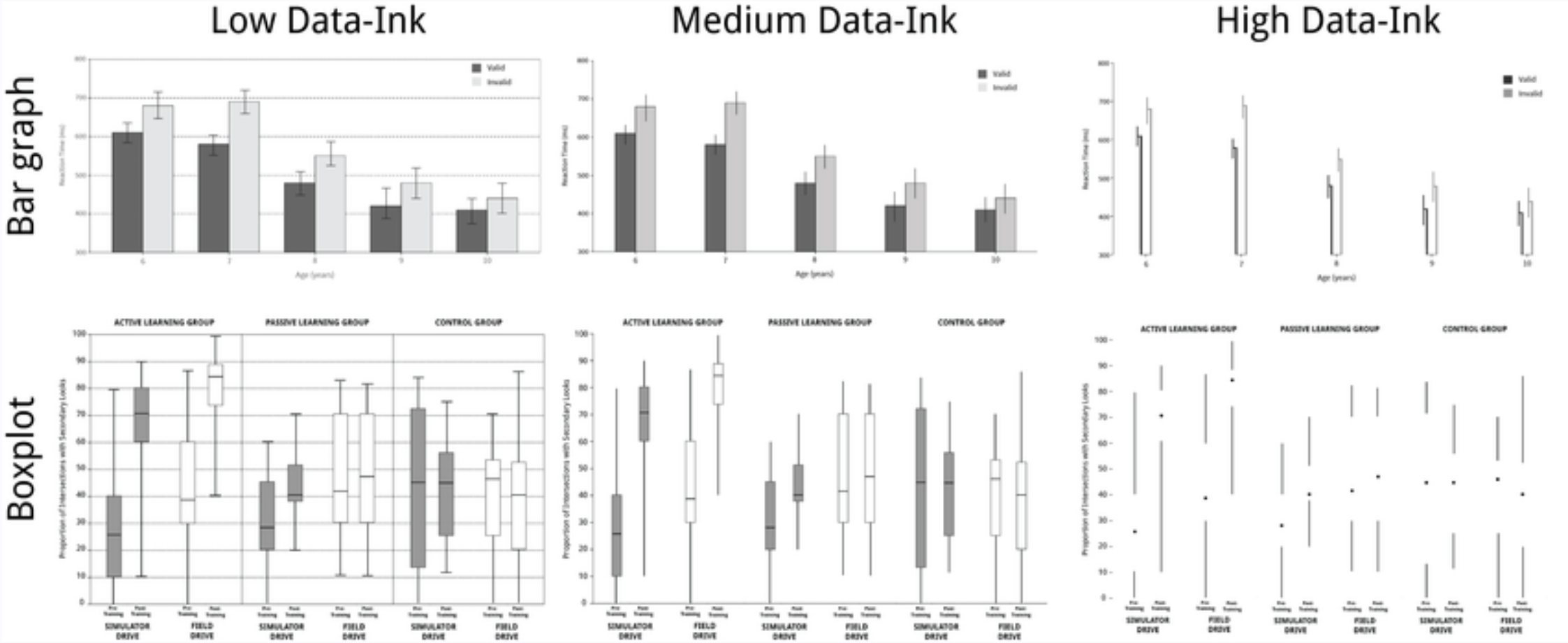
Figure 3: Exploration and adaptation behavior. (a) Preferred condition (n=30). (b) Mean slider adjustments (n=270 per condition); error bars show standard deviation (c) Point selection usage by device in C3 (n=90 per device).

Data-Ink Ratio

Opfundet af Edward Tufte. Hvor meget af det "blæk" I bruger på skærmen, viser rent faktisk data?

- **Slet støjen:** Fjern tunge baggrunde, unødvendige grid-linjer i baggrunden, kasser, rammer og 3D-effekter.
- **Brug farver strategisk:** Brug én neutral farve til alle normale data-barer, og brug jeres primære "Designsystem-farve" til at fremhæve den ene søjle, brugeren skal fokusere på.

Data-Ink Ratio



https://www.researchgate.net/figure/Experimental-stimuli-low-medium-and-high-data-ink-bar-graph-and-boxplots_fig5_307953810

Øvelse 1.1

Tænk tilbage på jeres valgte datasæt. For at kunne bygge en skarp "Happy Path", skal I kende slutdestinationen. Besvar følgende:

- **Kerneindsigten:** Hvad er det absolut vigtigste spørgsmål, brugeren skal have svar på via jeres data?
- **Værdien:** Hvorfor er dette svar vigtigt for brugerens hverdag?
- **Adfærden:** Hvilken handling håber I, at brugeren foretager, når de har set grafen?

Beskriv the happy path for jeres app / snak 2 og 2 fra forskellige udviklingsgrupper / 10 minutter



Øvelse 1.2

Skiftes til at tegne og præsentere jeres hurtige skitser for hinanden. Tag skitsen af jeres egen app med jer!

- **Tegn makkerens app:** Baseret *kun* på det, du lige har hørt: Hvordan ville du designe et interface og en graf, der guider brugeren direkte frem til kerneindsigten?
- **Fokus:** Hvilken type plot (bar, line, radar) giver bedst mening for deres data? Hvilke knapper er der brug for?

Tegn og fortæl / snak 2 og 2 / 10 minutter



Øvelse 1.3

Konkretiser jeres happy path I udviklings teams ved at diskutere indsigter fra øvelse 1.1 og 1.2

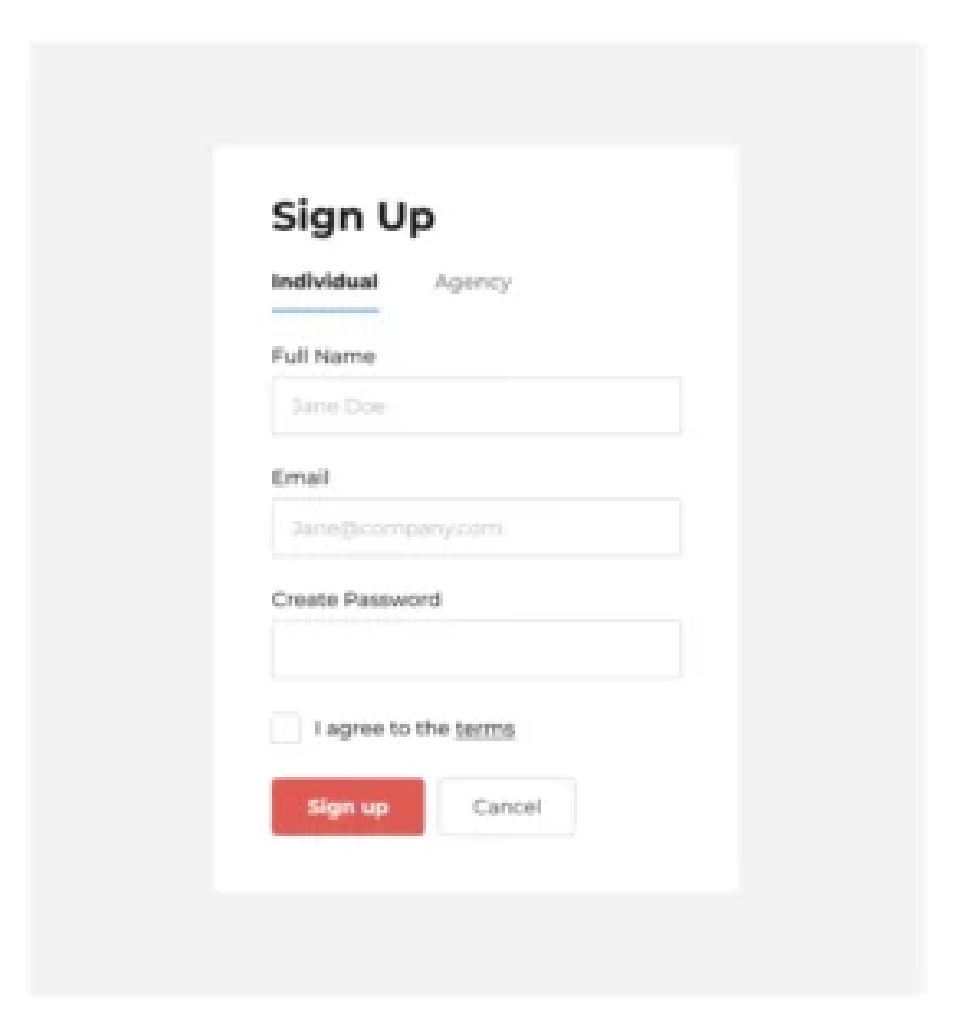
- remvis de skitser, de andre tegnede til jer. Gav deres ufiltrerede perspektiv jer nogle nye, simplere idéer til jeres InfoVis?
- Sammenlign skitserne med jeres nuværende Figma-design. Har I bygget for meget "Scope Creep"?
- Fastlæg appens endelige "Happy Path" (den mest direkte rute fra app-åbning til data-indsigt).

Walk and talk / i udviklings grupperne / 15 minutter



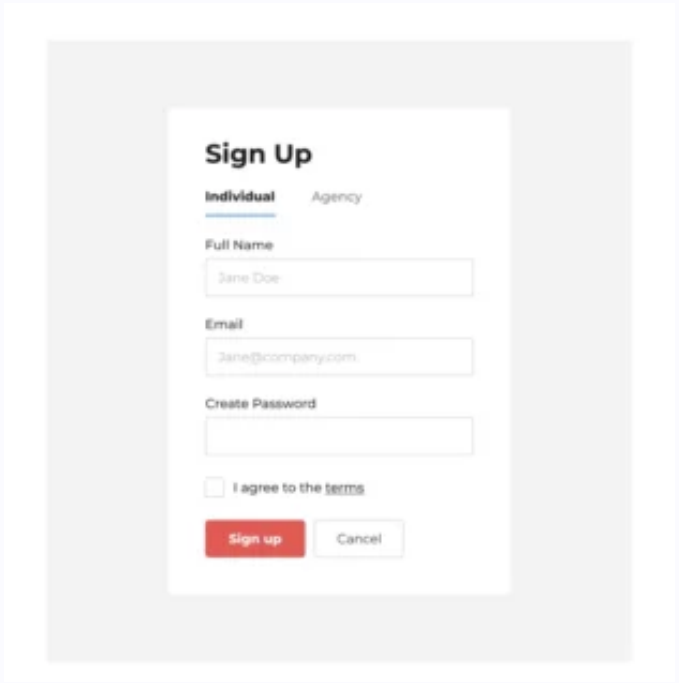
Designsystemer

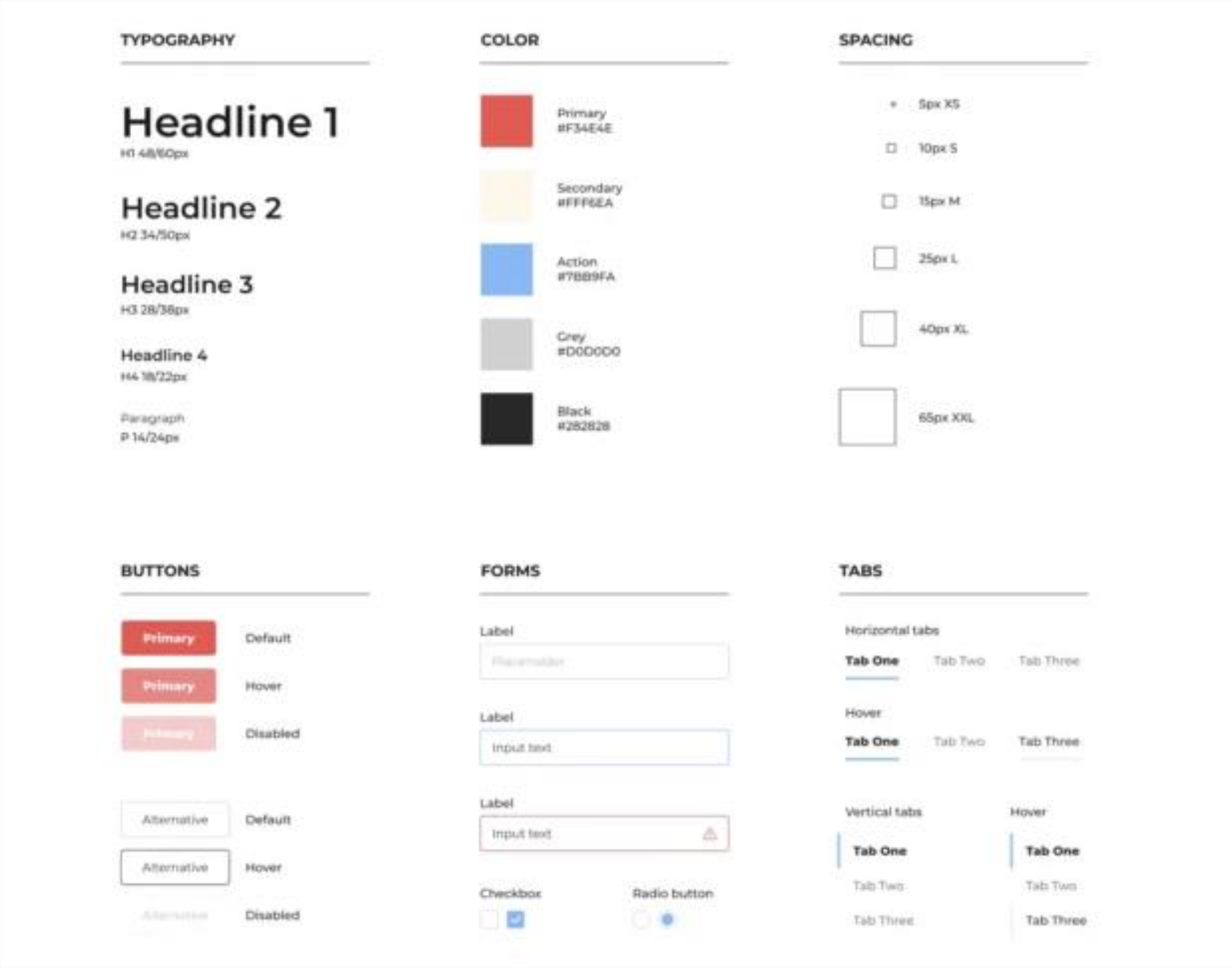
Designsystem



https://yeswebdesignstudio.com/blog/7-practices-for-figma-design-system/

Designsystem





https://yeswebdesignstudio.com/blog/7-practices-for-figma-design-system/

Designsystem

Konsistens og kvalitet

Ensartethed på tværs af produkter og platforme, færre design fejl

Effektivitet og skalering

Genbrugelige komponenter, hurtigere udvikling

Forenkler samarbejde

Fælles sprog mellem de forskellige teams

Fremtidssikret design

Nemmere vedligeholdelse og adoption af nye teknologier

https://yeswebdesignstudio.com/blog/7-practices-for-figma-design-system/

Designsystem

Farver

primær, sekundær, neutral, statusfarver(fejl,succes)

Typografi

skrifttype, størrelser, linjehøjde, hierarki

Ikoner & illustration

stil, filformat

Mellemrum & layout

grid system, afstandstokens

Komponenter

knapper, formularer, navigation

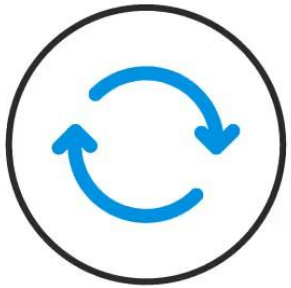
Retningslinjer

tilgængelighed, stil, tone

https://yeswebdesignstudio.com/blog/7-practices-for-figma-design-system/

Agile og Design thinking

Similarities between Design Thinking and Agile Methodologies



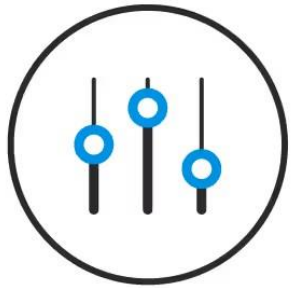
Iterative



User-Centered



Collaboration and Teamwork



Flexibility and Adaptability

Interaction Design Foundation
interaction-design.org

https://www.interaction-design.org/literature/topics/design-thinking

	Design Thinking	Agile
Oprindelse	Stammer primært fra design og låner fra flere discipliner, herunder psykologi, systemtænkning og forretningsstrategi.	Stammer primært fra softwareudvikling og låner fra discipliner såsom produktion og projektledelse.
Primært fokus	Problemløsning og innovative løsninger.	Effektiv produktlevering.
Anvendelsesfase	Typisk i begyndelsen af et projekt. Formålet er at definere problemet samt teste og vælge en løsning.	Typisk efter at teams har en klar løsning. Formålet er at levere den løsning og løbende iterere på det 'live' produkt.
Struktur og Dokumentation	Flydende proces, mindre formel og relativt mindre dokumentation.	Struktureret og formel proces med omfattende dokumentation.
Slutprodukt	En idé eller løsning, typisk med en prototype; er muligvis ikke håndgribelig.	Håndgribeligt, fungerende produkt (typisk software) leveret til slutbrugere.

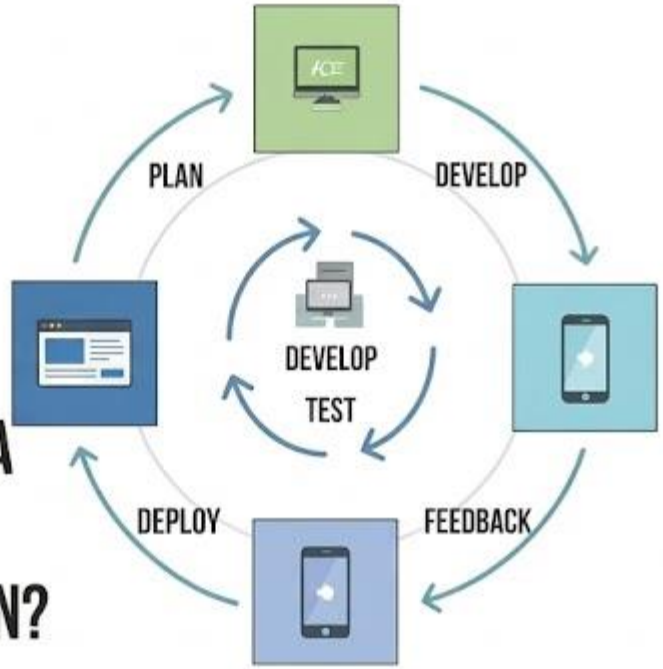
DESIGN THINKING



?

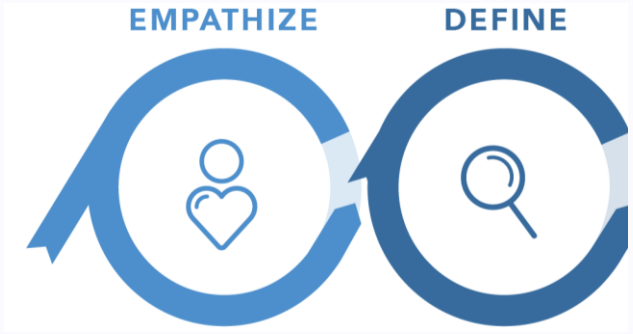
HVØRDAN GÅR VI FRA ABSTRAKT IDÉ TIL KONKRET PRODUKTION?

AGILE

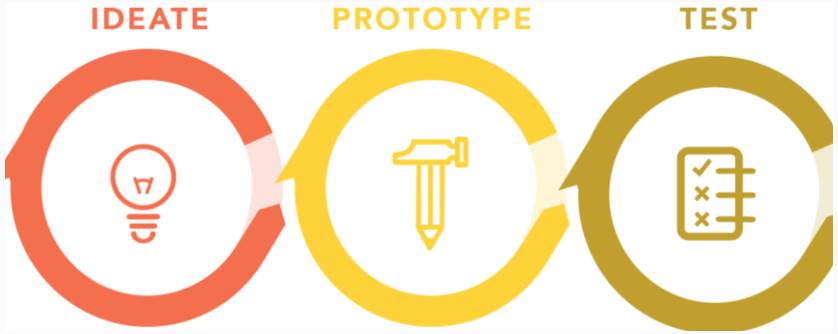


Design thinking 101

- **Hvad** er det rigtige problem



- **Hvad** er den rigtige løsning



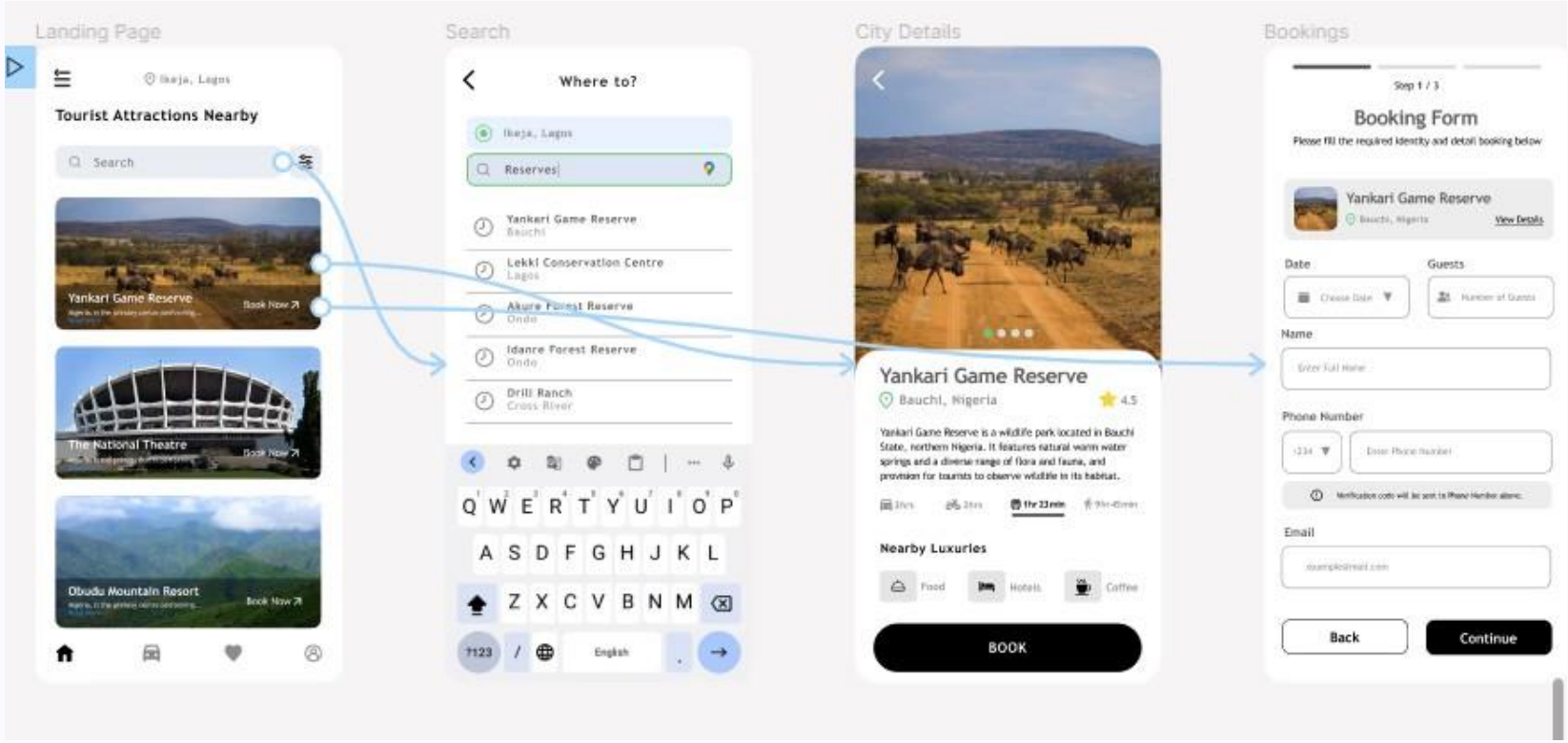
Agil udvikling

- **Hvordan** bygger vi løsningen



User Test

Bruger mål



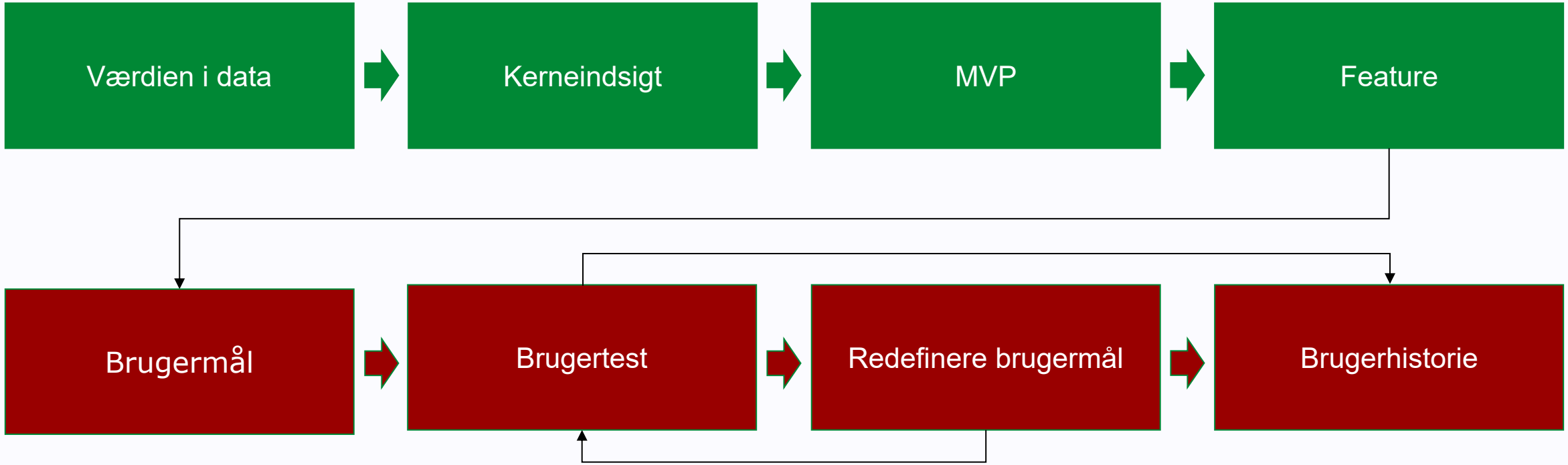
https://www.banani.co/definitions/figma-prototype

User Test udfald

- Præcisering af hvordan en bruger udfører et brugermål (undgår scope creep)
- Målrettet interaktionen (risikostyring)
- Konkretisering af påkrævede ændringer (analyse og refleksion)

https://www.banani.co/definitions/figma-prototype

Fra happy path til User story

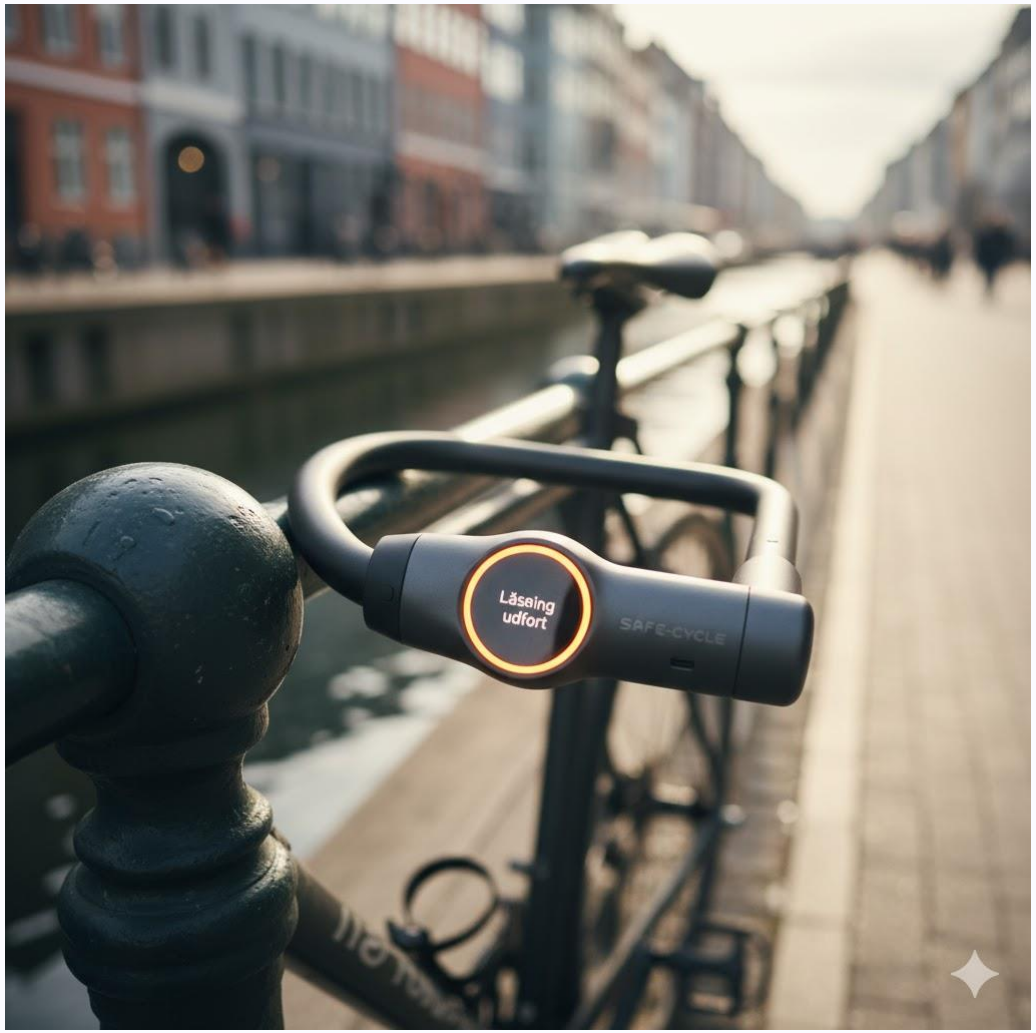


https://www.banani.co/definitions/figma-prototype

Fra happy path til User Story

Brugermål:

Lås din cykel med låsen



Fra happy path til User Story

Brugermål:

Lås din cykel med låsen

Observation fra bruger test

- Brugerne tjekker flere gange om låsen er låst
- Brugeren tænker højt:

”jeg må hellere lige være sikker på, om den *faktisk* var låst”



Fra happy path til User Story

Brugermål:

Lås din cykel med låsen

Observation fra bruger test

- Brugerne tjekker flere gange om låsen er låst
- Brugeren tænker højt:

”jeg må hellere lige være sikker på, om den *faktisk* var låst”

Behov: at føle sig sikker på cyklen ikke stjæles

Frustration: Det er ikke klart for brugeren at låsen er låst



Fra happy path til User Story

Brugermål:

- Lås din cykel med låsen
- Behov: at føle sig sikker på cyklen ikke stjæles
- Frustration: Det er ikke klart for brugeren at låsen er låst

User Story:
Skabelon: "Som en [**TYPE AF BRUGER**], vil jeg [**EN HANDLING**], så [**EN VÆRDI ELLER GRUND**]."

Forslag:

"Som en **cykelejer, der lige har forladt sin cykel**, vil jeg **se en klar 'status'-indikator (f.eks. 'LÅST' eller 'ULÅST'),** så **jeg har ro i sindet og ved med sikkerhed, at min cykel er tryg**."



Øvelse 2.1

Kig på de skitser, I fik tegnet i Øvelse 1.

- Vælg **én specifik interaktion eller del af skærmen** fra skitsen (f.eks. filtreringsmenuen, selve grafvisningen, eller en details-on-demand popup).
- Skriv en afgrænset Agil User Story for netop dén del. Brug skabelonen:
  - **"Som en [TYPE AF BRUGER], vil jeg [SPECIFIK HANDLING PÅ SKITSEN], så [DEL-MÅLET/VÆRDIEN]."** *(Arbejd i jeres udviklingsteams / 5 minutter)*

Alene / 5 minutter



Øvelse 2.2

Find sammen med den person fra den anden gruppe, som tegnede den oprindelige skitse til jer.

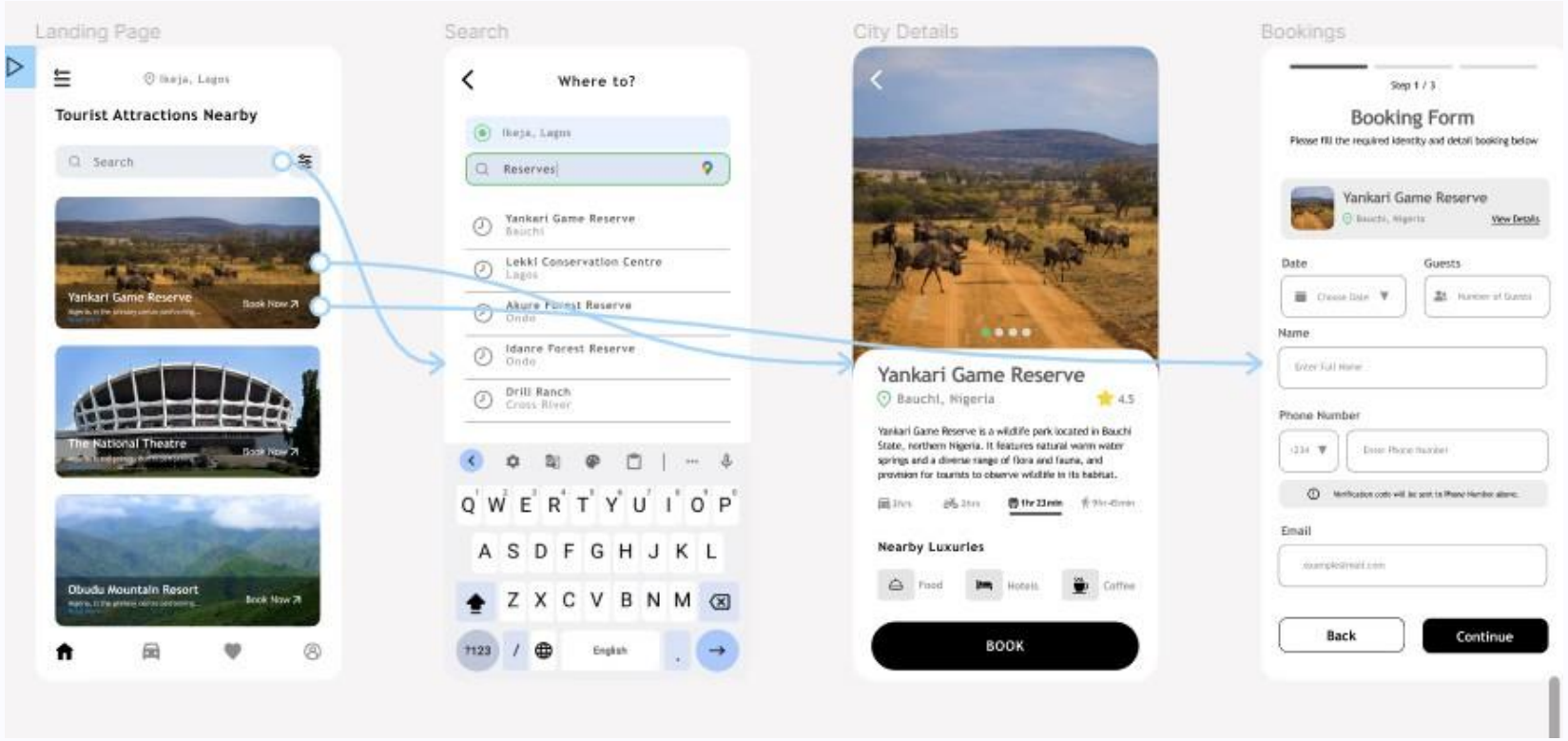
- Læs jeres nye User Story højt for dem.
- **Designerens opgave:** Fanger denne User Story intentionen bag lige netop dét element på skitsen?
- **Udviklerens opgave (Stresstest):** Er opgaven lille og konkret nok til at kunne kodes på et par dage? Eller kom I uforvarende til at skrive et helt *Epic* ned, som skal brydes ned i endnu flere små User Stories?

Snak 2 og 2 / 10 minutter



User Test

User Story +



https://www.banani.co/definitions/figma-prototype

Acceptancekriterier

•**De er verificerbare krav:** ACs beskriver systemets adfærd under forskellige omstændigheder. De er skrevet så præcist, at de kan bruges direkte som testcases.

•**De er fra brugerens perspektiv (men teknisk præcise):** Selvom de beskriver systemets adfærd, er de stadig bundet til den værdi, User Story'en leverer. De definerer, hvordan brugeren (eller systemet) *ved*, at User Story'en er opfyldt.

•**De skaber klarhed:** ACs fjerner tvivl og misforståelser mellem product owner, udviklere og testere. Alle ved præcis, hvad der skal bygges, og hvordan succes måles.

•**De reducerer risiko og re-work:** Ved at specificere 'Done' på forhånd undgår man, at udviklere bygger noget, der ikke lever op til forventningerne, hvilket sparer tid og ressourcer på ‘re-work'. Det er *kravspecifikation på mikro-niveau*.

"Givet-Når-Så" (Gherkin-syntaks)

Den mest almindelige og anbefalede skabelon for ACs er "Givet-Når-Så" (på engelsk kendt som Gherkin eller BDD - Behavior-Driven Development). Dette format tvinger os til at tænke i konkrete scenarier:

•**Givet (Given):** Beskriver den **forudsætning** eller den **begyndelsestilstand**, systemet befinder sig i. Hvad er konteksten?

•**Når (When):** Beskriver den **specifikke handling** eller den **udløsende begivenhed**, der finder sted. Hvad sker der?

•**Så (Then):** Beskriver det **observerbare resultat** eller den **forventede adfærd** fra systemet. Hvad skal der ske?

Eksempler

"Som en **cykelejer, der lige har forladt sin cykel**, vil jeg **se en klar 'status'-indikator (f.eks. 'LÅST' eller 'ULÅST'),** så **jeg har ro i sindet og ved med sikkerhed, at min cykel er tryg**."

Acceptance Kriterier:

•AC 1 (Positivt flow):

- **Givet** at app'en er åben OG har Bluetooth-forbindelse til låsen.
- **Når** låsen fysisk lukkes og bekræfter 'låst'-status.
- **Så** skal status-ikonet i app'en skifte til 'LÅST' (grønt) inden for 2 sekunder.

•AC 2 (Negativt flow / Fejlhåndtering):

- **Givet** at app'en er åben OG har Bluetooth-forbindelse til låsen.
- **Når** låsen forsøger at lukke, men fejler (f.eks. pga. et blokerende objekt).
- **Så** skal en fejlbesked ('Låsning fejlede: Tjek for blokering') vises i app'en, og status-ikonet skal forblive 'ULÅST' (rødt).

•AC 3 (Kanttilfælde / Edge Case):

- **Givet** at app'en er åben og viser 'LÅST' (grøn) status.
- **Når** Bluetooth-forbindelsen til låsen pludselig mistes.
- **Så** skal status-ikonet ændre sig til 'Offline (sidst set: [tidspunkt])', og ikonet skal være gråt.

Øvelse 3

Find på acceptance kriterier for den "Happy Path" (User Story), I netop har defineret for jeres egen app. Brug formen:

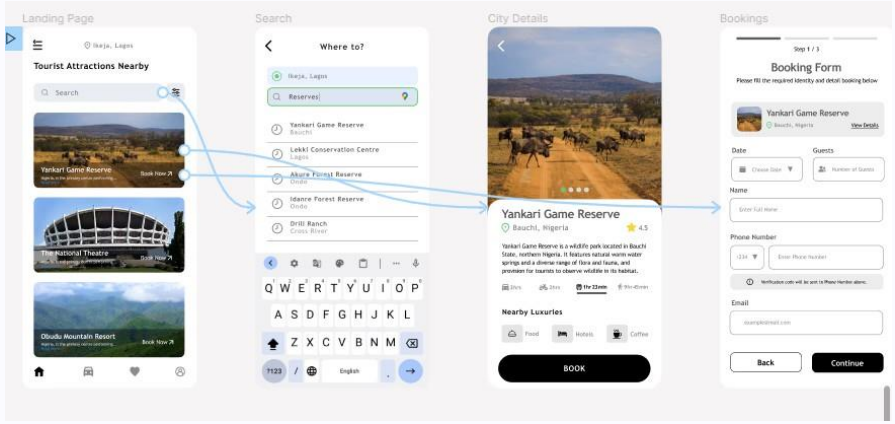
- **Givet (Given):** Beskriver den **forudsætning** eller den **begyndelsestilstand**, systemet befinder sig i. Hvad er konteksten?
- **Når (When):** Beskriver den **specifikke handling** eller den **udløsende begivenhed**, der finder sted. Hvad sker der?
- **Så (Then):** Beskriver det **observerbare resultat** eller den **forventede adfærd** fra systemet. Hvad skal der ske?

Sammen 2 og 2 / samme som tidligere / 15 minutter



User Test udfald

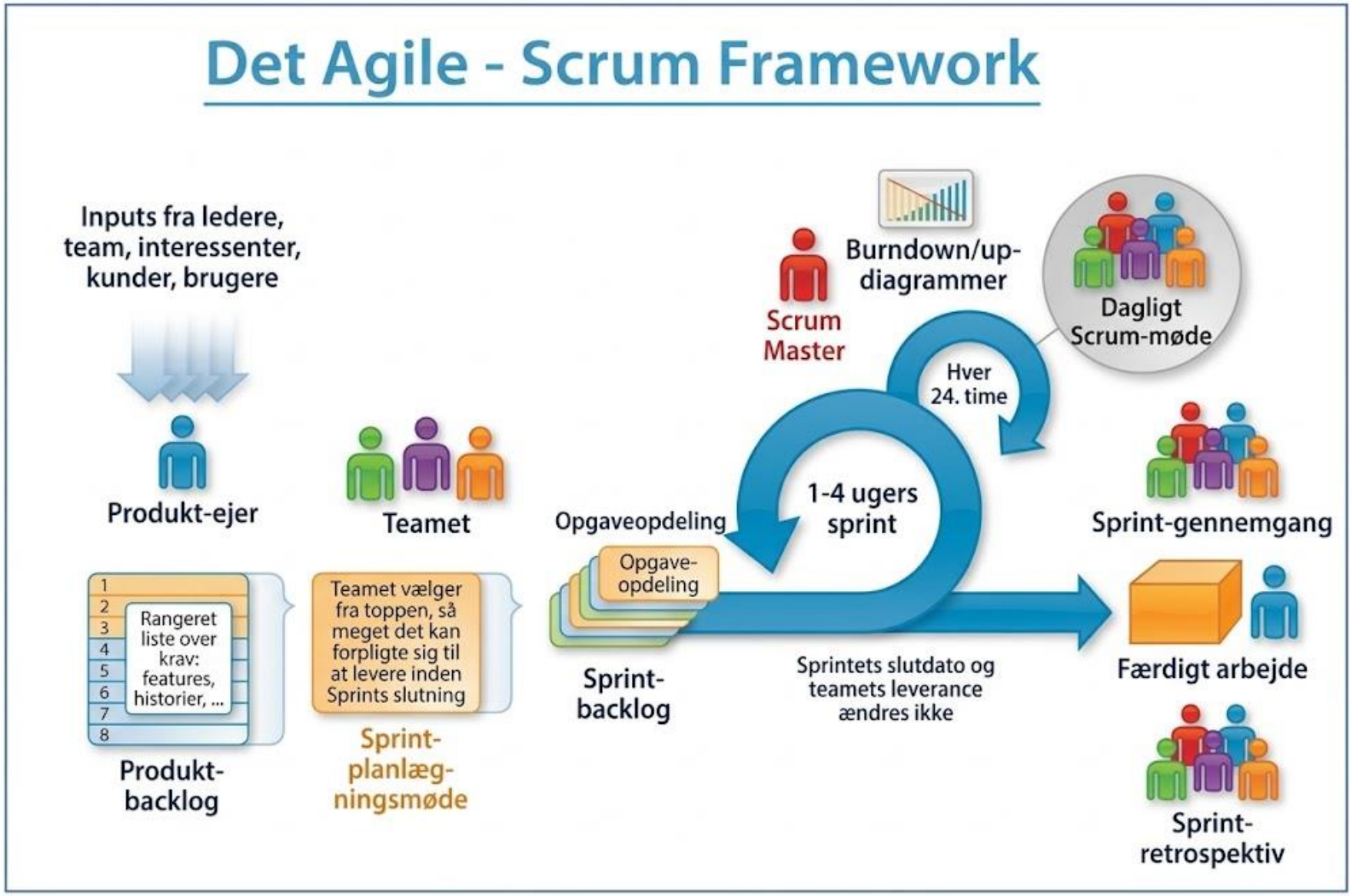
User Story + Acceptance Kriterier +



https://www.banani.co/definitions/figma-prototype

PAUSE

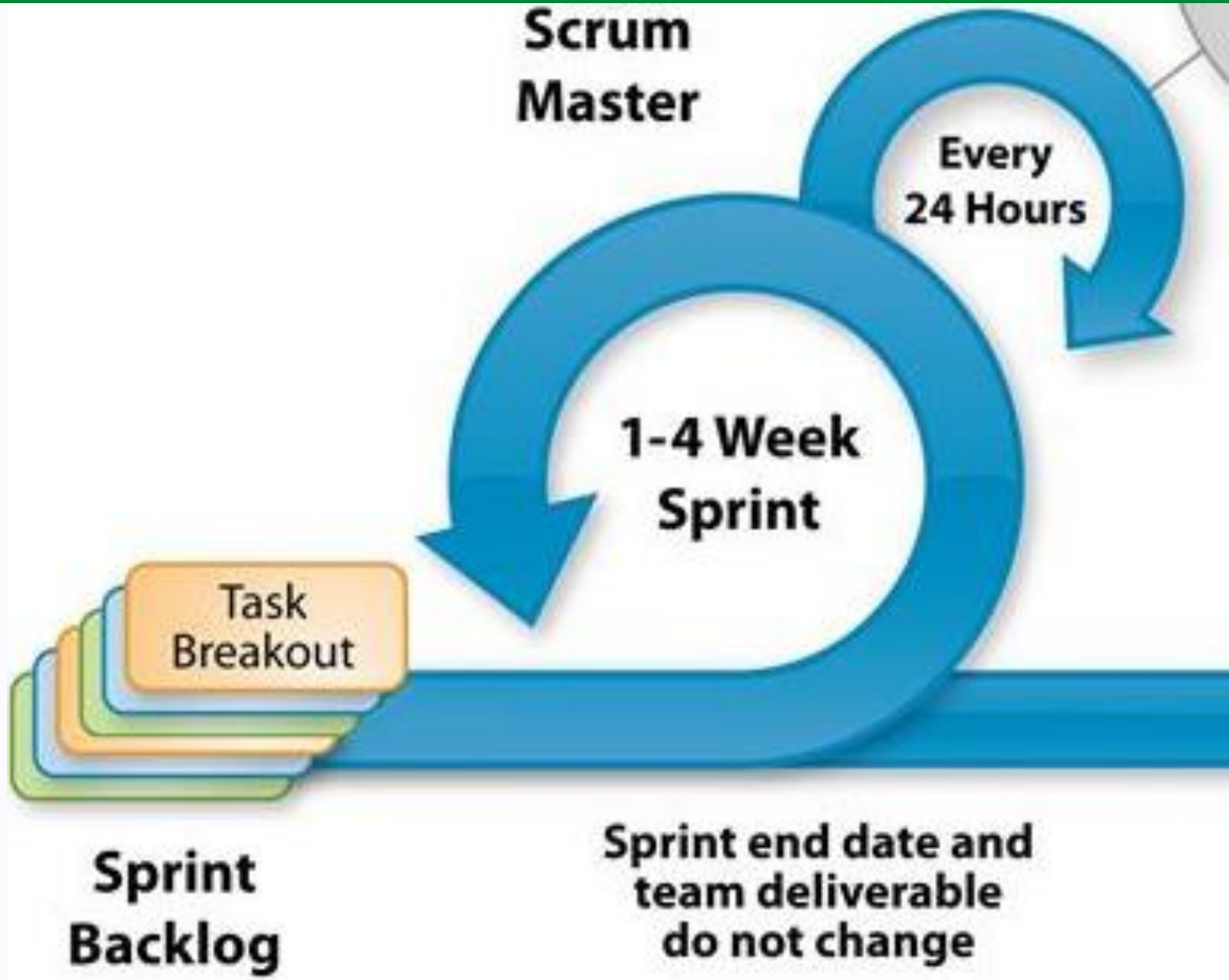
Agile udvikling



https://www.c-sharpcorner.com/UploadFile/d9c992/the-agile-scrum-framework/

Agil udvikling

- Cykler på 1-4 uger
- Iterativ implementering af vigtigste features
- Mulighed for at tilpasse udviklingen løbende
- Kan øge produktiviteten når processen kører godt



https://www.c-sharpcorner.com/UploadFile/d9c992/the-agile-scrum-framework/

Agil udvikling

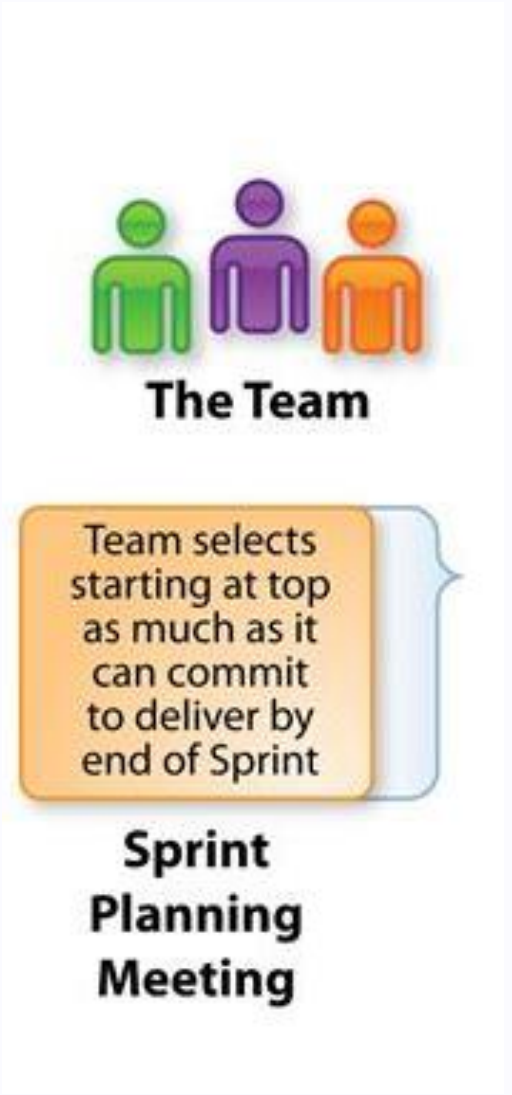
- En liste over alle de ting der skal implementeres i produktet
- Features kan være mere eller mindre gennemarbejdet



https://www.c-sharpcorner.com/UploadFile/d9c992/the-agile-scrum-framework/

Agil udvikling

- Møde med product owner
- Til prioritering af hvad der skal arbejdes på i løbet af næste sprint
- Vigtigt at ticket der vælges er veldefineret og gennemarbejdet



https://www.c-sharpcorner.com/UploadFile/d9c992/the-agile-scrum-framework/

Agil udvikling

- Korte møder
- Synkroniser holdet
- Giver muligheden for at spørge om hjælp
- Giver muligheden for at komme med forslag
- Virker kun med små tickets



https://www.c-sharpcorner.com/UploadFile/d9c992/the-agile-scrum-framework/

Agil udvikling

- Fokus på tickets
- Burn down chart
- Undersøgelse af tickets der ikke gik efterplanen
- Produkt fremvisning for product owner



https://www.c-sharpcorner.com/UploadFile/d9c992/the-agile-scrum-framework/

Agil udvikling

- Fokus på udviklerne
- Touchy feely
- Formålet er at forbedre samarbejde
- Ikke dømmende rum hvor problemer og irritationer kan diskuteres



https://www.c-sharpcorner.com/UploadFile/d9c992/the-agile-scrum-framework/

Sprint Reviews



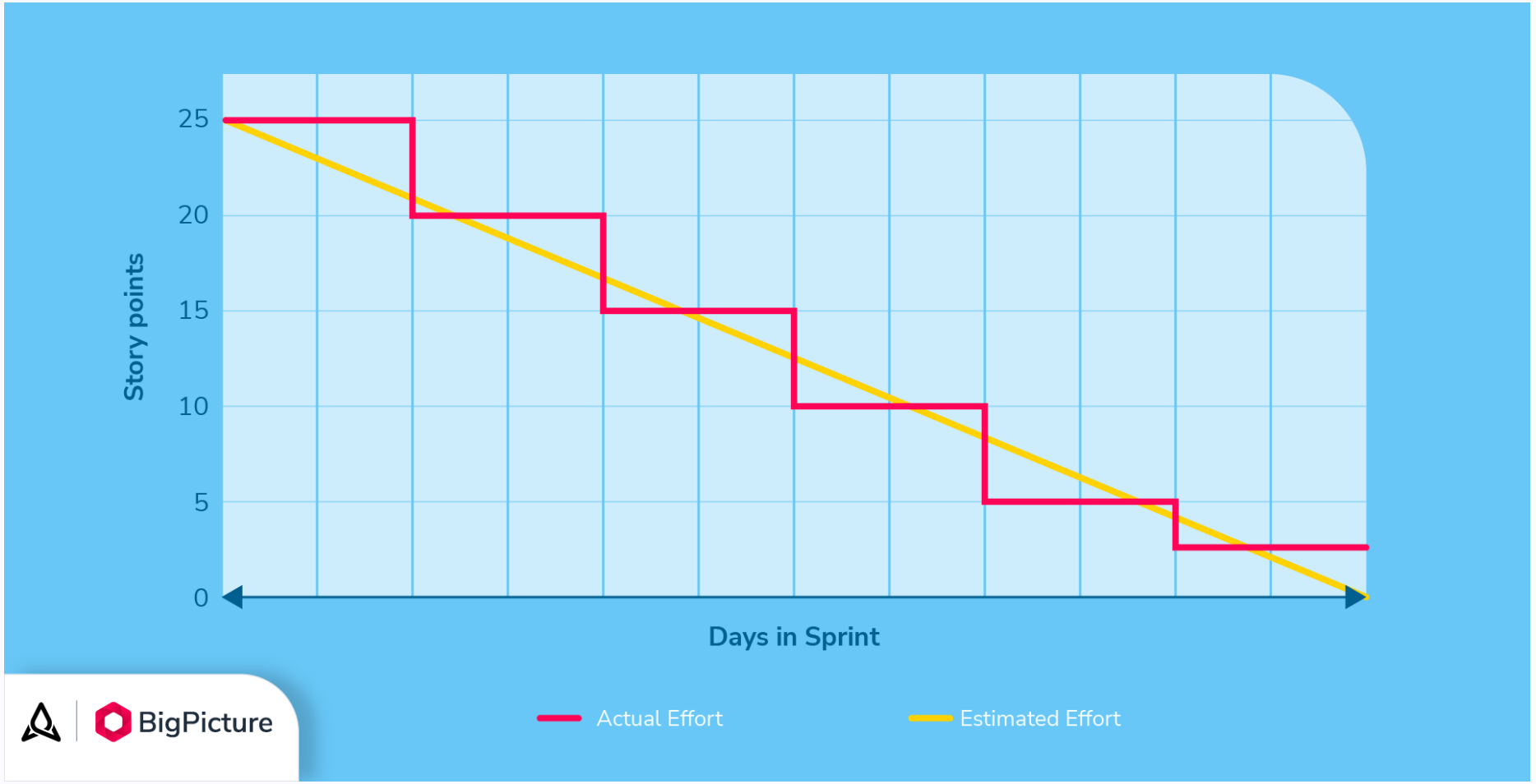
- Efter hver sprint i 3 ugers perioden laves en sprint rapport som inkluderer burndown chart samt en lille beskrivelse af hvordan applikationen er blevet udviklet i løbet af sprintet. Refleksioner over potentielle blokeringer og hvordan de kan fjernes inkluderes.
- En skabelon vil være tilgængelig på learn inden tre ugers perioden

Sprint Reviews



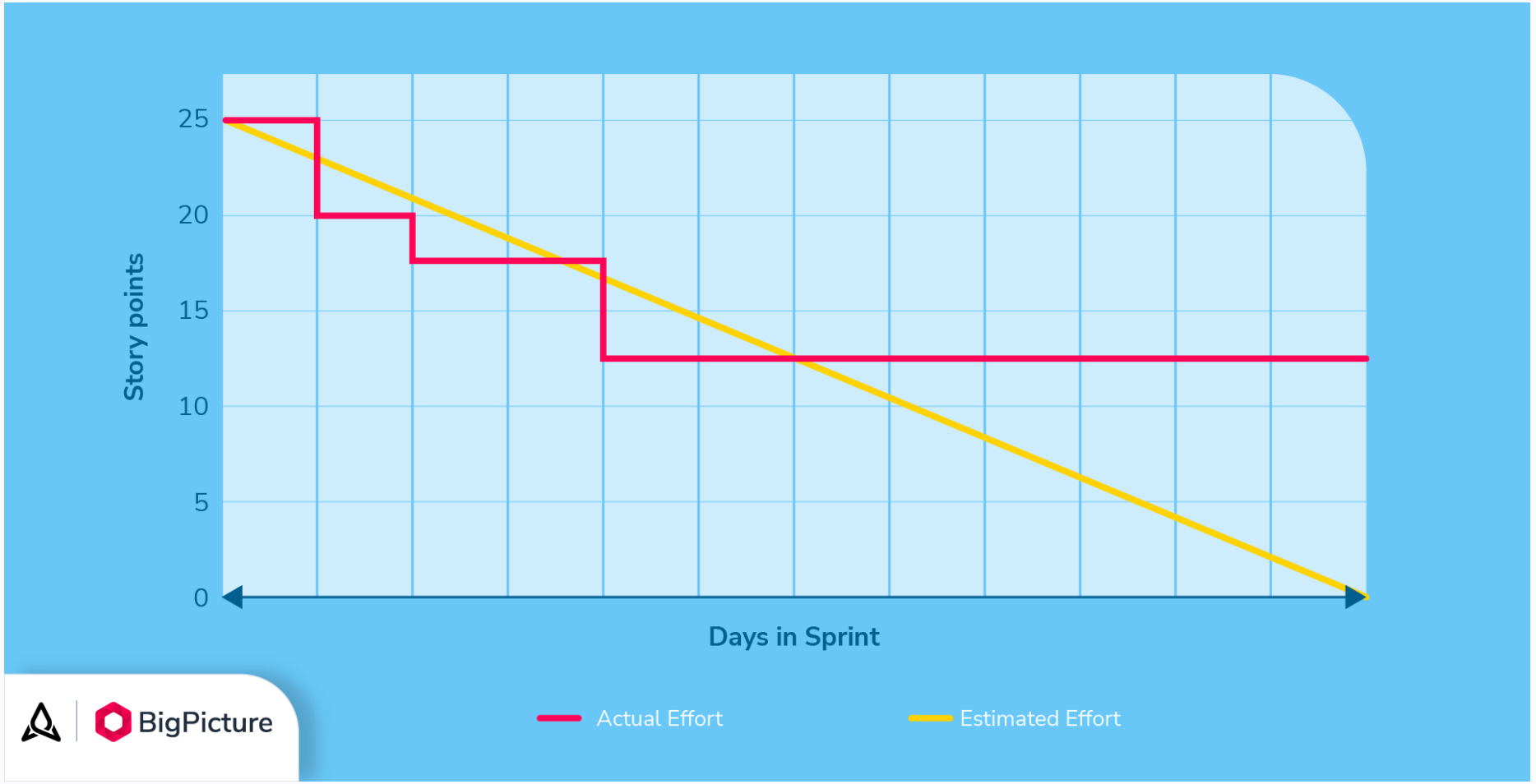
- Hvad er nået ?
- Er der nogle produktivitets-blocker?
- Burn down chart
- Passede estimeringen af tickets?
- Næste sprints tickets + estimering

Burn down chart



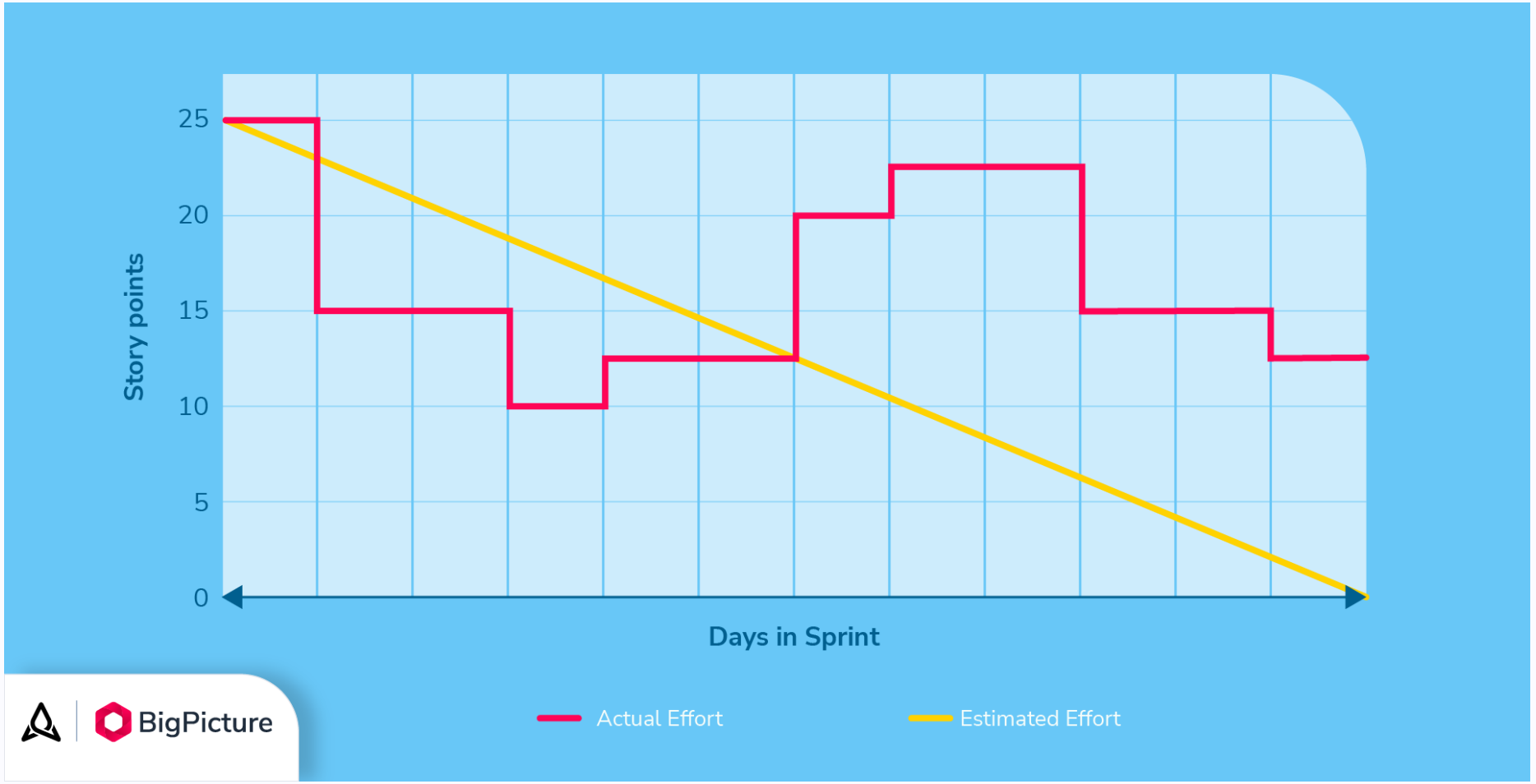
- https://bigpicture.one/blog/sprint-burndown-chart-101/

Burn down chart

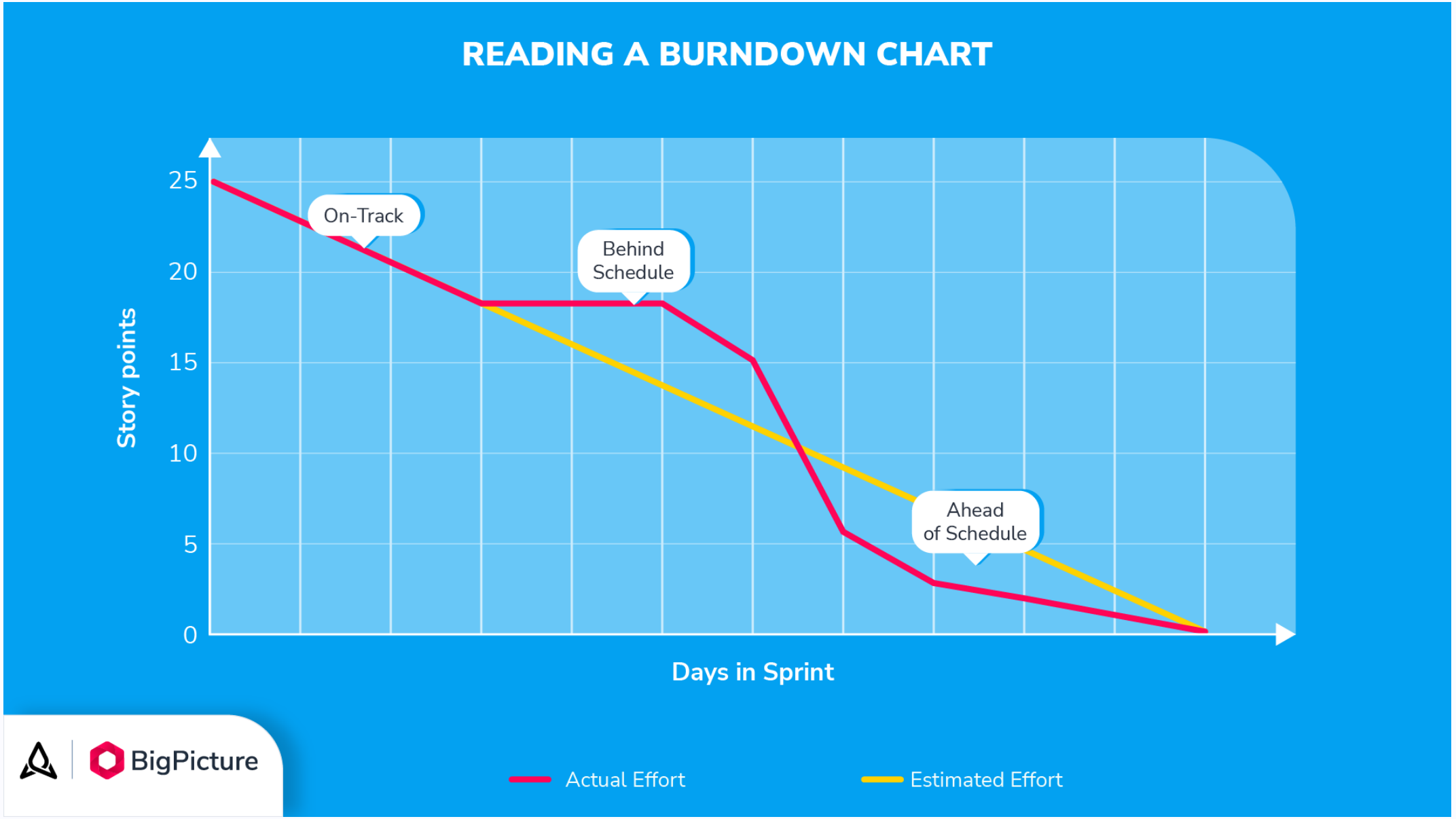


- https://bigpicture.one/blog/sprint-burndown-chart-101/

Burn down chart



- https://bigpicture.one/blog/sprint-burndown-chart-101/

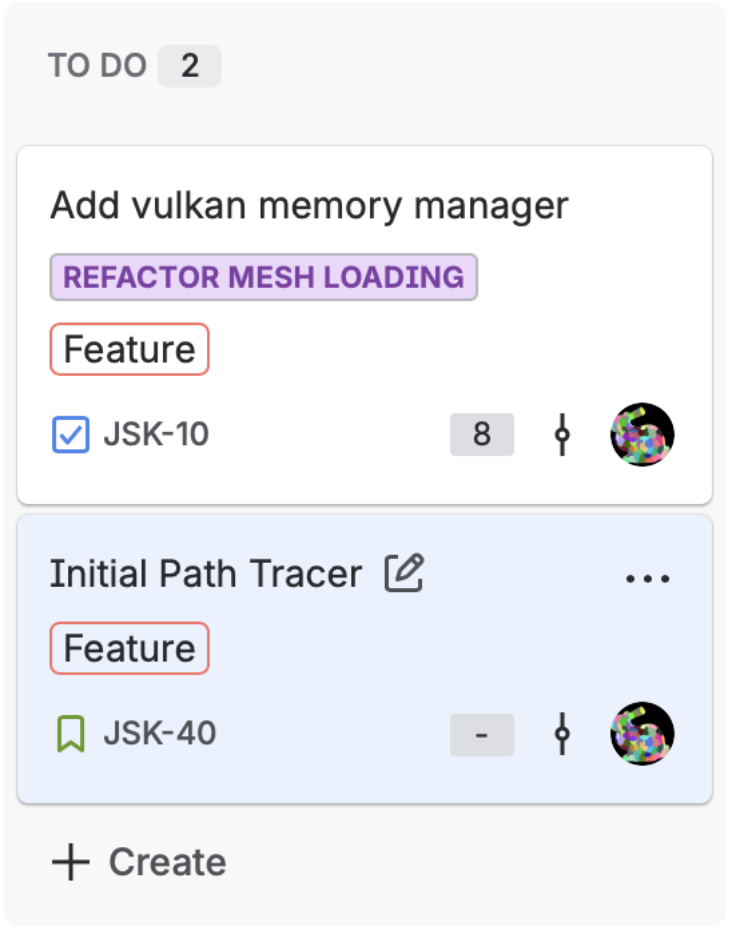


- https://bigpicture.one/blog/sprint-burndown-chart-101/

Tickets

Tickets

- Et **ticket** (eller "issue", "item") er den mindste, atomare enhed af arbejde i et agilt system (som Jira, Trello, etc.).
- Det er en enkelt opgave, der skal spores, tildeles og fuldføres.
- Alt arbejde – fra en simpel opgave til en kompleks fejl – starter som et ticket.
- Tickets sikrer **transparens** og **sporbarhed** i teamets workflow.



Fra småt til Stort

De fleste agile teams bruger et hierarki til at organisere arbejde:

- **Epic** (Det store mål)
  - **Feature** (En brugbar del af Epic'et)
    - **User Story** (Et specifikt brugerbehov)
      - **Ticket/Sub-task** (En teknisk delopgave)

Side-items:

- **Bug** (Fejl),
- **Spike** (Undersøgelse)

Refactor mesh loading

Description

Attachments 1

Name	Size	Date added
 billede-20250310-091927.png	2.9 MB	10 Mar 2025 10:24am

Child work items

0% Done

Work	Pri...	As...	Status
JSK-14 Refactor tinyobjloader out of meshletmaker	= M	l	TO DO
JSK-13 Consider serializing the meshes in the post loaded format to imporve on startup time	= M	l	TO DO
JSK-12 Add texture Loading to Resource Manager	= M	l	TO DO
JSK-11 Add Obj Loading to Resource Manager	= M	l	TO DO
JSK-22 support glTF	= M	l	TO DO
JSK-10 Add vulkan memory manager	= M	r	TO DO

Epics og Features

- **Epic:**
  - Et meget stort stykke arbejde eller et strategisk mål, der er for komplekst til et enkelt sprint.
  - Brydes ned i mindre dele over tid.
  - *Eksempel:* "Implementering af ny betalingsløsning".
- **Feature:**
  - En funktionel del af produktet, der leverer specifik værdi til brugeren.
  - Består ofte af flere User Stories.
  - *Eksempel:* "Mulighed for at betale med MobilePay".

Bugs og Tickets

- **Bug (Fejl):**
  - En fejl eller defekt i **eksisterende** funktionalitet.
  - Noget virker ikke, som det var hensigten.
  - *Eksempel:* "Brugeren kan ikke logge ind med korrekt password."
- **Ticket (Opgave):**
  - En specifik, teknisk handling, der skal udføres.
  - Ofte en "sub-task" til en User Story.
  - *Eksempel:* "Opsæt database-tabel til brugerprofiler" eller "Skriv unit tests til login-flow".

Spikes og ”sub-tasks”

- **Spike (Undersøgelse):**
  - Et tidsbegrænset ticket, hvis formål er at **undersøge** noget og **reducere usikkerhed**.
  - Bruges til at research en ny teknologi, forstå en kompleks forretningsregel eller afdække tekniske risici.
  - *Resultat:* Ikke fungerende kode, men **viden** (f.eks. en rapport, et diagram, et "ja/nej" svar).
- **Sub-task (Delopgave):**
  - En yderligere nedbrydning af en Story eller Bug i mindre, håndgribelige trin.
  - Typisk tekniske opgaver (f.eks. "Opret database-felt", "Byg UI-komponent", "Opdater API-endpoint").
  - **Vigtigt:** Sub-tasks **testes ikke enkeltvis**. De er så små, at de først giver mening og testes samlet som en del af deres "forælder"-ticket (f.eks. User Story'en).

Fra stort til småt

- **Refinement** (også kaldet *Backlog Grooming*) er en løbende proces, **ikke** kun et enkelt møde.
- **Formål:** At gennemgå, diskutere, prioritere og nedbryde tickets fra produkt-backloggen.
- **Mål:** At sikre, at de øverste emner i backloggen er "Ready" til næste sprint.
- Et "Ready" ticket er:
  - **Klart** (alle forstår, *hvad* der skal laves)
  - **Testbart** (har klare acceptkriterier)

Ready for Refinement vs Definition of Ready

Ready for Refinement (Klar til diskussion):

- **Hvem:** Product Owner og Designer
- **Kriterier:**
  - Der er et klart **formål** (Hvorfor gør vi dette?).
  - **Værdien** for brugeren eller forretningen er beskrevet.
  - Omfanget er nogenlunde afgrænset (f.eks. knyttet til et Epic).
- *Mål:* Ticket'en giver mening at *begynde* at diskutere med teamet.

Ready for Refinement vs Definition of Ready

Definition of Ready (Klar til Sprint):

- **Hvem:** Hele teamet bliver enige om dette *under* refinement.
- **Kriterier (eksempler):**
  - Story'en er estimeret (har Story Points).
  - Acceptkriterier er skrevet og forstået.
  - UI/UX-design er vedhæftet (hvis relevant).
  - Afhængigheder (tekniske/eksterne) er afklaret.
  - Lille nok (kan færdiggøres i ét sprint)
- *Mål:* Ticket'en kan trækkes direkte ind i et sprint *uden* yderligere spørgsmål.

Product Refinement Ceremoni

- **Fokus: HVAD** og **HVORFOR**.
- **Deltagere:** Product Owner (ejer mødet), udviklingsteamet, designere, evt. interessenter.
- **Aktiviteter:**
  - Product Owner præsenterer tickets, der er "Ready for Refinement".
  - Teamet stiller spørgsmål for at forstå *formålet* og *værdien*.
  - Udarbejdelse og præcisering af acceptkriterier.
  - *Mål:* At flytte tickets mod "Definition of Ready".



Product Refinment Ceremony

Technical Refinement Ceremoni

- **Fokus: HVORDAN**.
- **Deltagere:** Primært udviklingsteamet (udviklere, testere, tech lead).
- **Aktiviteter:**
  - Diskutere teknisk implementering (datamodeller, API'er, arkitektur).
  - Identificere afhængigheder (til andre teams, systemer).
  - Afdække tekniske risici eller usikkerheder.
  - Nedbryde stories i mindre (tekniske) sub-tasks, hvis nødvendigt.
  - *Resultat:* Teamet har en fælles forståelse af, *hvordan* opgaven skal løses.



Ticket Estimering

- **Formål:** Ikke at fastsætte en deadline, men at skabe **fælles forståelse**.
- Estimering tvinger teamet til at diskutere en opgaves **kompleksitet**.
- **Det hjælper med at:**
  - Afdække skjulte antagelser ("Jeg troede, vi bare...")
  - Identificere usikkerhed og risici.
  - Skabe en prognose for, hvor meget arbejde teamet kan nå (se Sprint Velocity).

Story Points og Planning Poker

Story Points er en **relativ** måleenhed. De mest almindelige er **Fibonacci-rækken** (1, 2, 3, 5, 8, 13...).

- Hvorfor? Afstanden mellem tallene vokser, hvilket afspejler, at usikkerheden stiger med kompleksiteten.

- **Planning Poker:**
  - En teknik til at nå konsensus om Story Points.
  - Alle i teamet "stemmer" (med kort) samtidigt.
  - Hvis der er stor spredning (en 2'er og en 13'er), diskuterer teamet *hvorfor*. Dette er **kernepunktet** – det er her, den tekniske refinement sker!

Absolut vs Relativ Estimering

- **Absolut Estimering (f.eks. Timer/Dage):**
  - *Hvordan:* "Denne opgave tager præcis 6 timer."
  - *Problem:* Mennesker er notorisk dårlige til at forudsige tid. Det skaber "buffer-tid" og fokuserer på *tid* i stedet for *værdi*.
- **Abstrakt/Relativ Estimering (f.eks. Story Points):**
  - *Hvordan:* "Denne opgave er *større* end Opgave A (3 points) men *mindre* end Opgave B (8 points). Vi giver den 5 points."
  - *Fordel:* Fokuserer på **Kompleksitet**, **Usikkerhed** og **Indsats** (Effort).



https://www.amazon.ca/Agile-Estimation-Poker-Cards-Fibonacci/dp/B01MDTJDST

Planning Poker: How-To

- **1. Etablér en Baseline:**
  - Bliv enige om en velkendt, let ticket (f.eks. "en 2'er" eller "en 3'er"). Dette **aligner**teamets skala.
- **2. Gennemgå Tickets (én ad gangen):**
  - Product Owner/Tech Lead præsenterer kort ticket'en (Hvad & Hvorfor).
  - Teamet stemmer samtidigt.
- **3. Håndter Spredning:**
  - Ved stor spredning (f.eks. 2, 5, 5, 13): Den med **højest** (13) og **lavest** (2) estimat forklarer deres tanker.
  - Teamet diskuterer kort og **stemmer igen**.
- **4. Vigtige Regler:**
  - **Stor ticket (f.eks. 13+):** For stor til ét sprint. Skal tilbage til refinement og **brydes ned**.
  - **Stor variance:** Et **hint** om, at ticket'en er uklart beskrevet. Den mangler sandsynligvis mere **Product Refinement** eller **Technical Refinement**.

Sprint Velocity

- **Velocity** er et **historisk** gennemsnit af, hvor mange Story Points et team *fuldfører*(accepteret som "Done") pr. sprint.
- *Eksempel:* Hvis teamet færdiggør 20, 22 og 18 points i de sidste 3 sprints, er deres velocity ca. 20.
- **VIGTIGT:** Velocity er et **planlægningsværktøj** (en prognose), *ikke* et performance-mål (KPI).
- Man kan *ikke* sammenligne velocity mellem to forskellige teams.

Sammenhængen

- **God Refinement** (baseret på "Ready for Refinement" tickets) fører til klare opgaver, der opfylder "Definition of Ready".
- **God Estimering** (f.eks. Planning Poker) tvinger til de vigtige diskussioner, der er nødvendige for at opnå "Definition of Ready".
- **Resultere i et stabil Sprint Velocity**.
- **Retrospective** og **stand ups** sørger for at teamet bliver bedre til at arbejde sammen

Opsummering

- **Tickets** (Epics, Stories, Bugs) organiserer alt vores arbejde.
- **Refinement** (Ticket & Technical) er processen, der flytter tickets fra "Ready for Refinement" til "Definition of Ready".
- **Estimering** (Story Points) er et værktøj til at skabe **fælles forståelse** af kompleksitet.
- **Velocity** er vores **prognoseværktøj**, der bygger direkte på kvaliteten af vores refinement.

Øvelse 4

Gå sammen i udviklingsgrupper

- Lav et udviklingsteam på Github Projects
- Opret jeres "Happy Path" som den første overordnede User Story (inkluder jeres netop definerede Acceptance Kriterier). definer acceptance kriterier
- Bryd denne User Story ned i konkrete tekniske og designmæssige *tickets* (f.eks. "Sæt navigationsbar op i Figma", "Forbered datasæt", "Opsæt Supabase projekt").
- Estimer jeres tickets i fællesskab med en Planning Poker mobile app.

